

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

MIARFID Official Master's Degree in Artificial Intelligence, Pattern Recognition and Digital Imaging

Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

FARM: A Formal Approach to Reward Machine Generation in Reinforcement Learning

MASTER FINAL WORK

Master in Artificial Intelligence, Pattern Recognition and Digital Image

Author: Miquel Gómez Corral

Tutor: Eva Onaindía De La Rivaherrera

Course 2026-2027

Resum

La construcció automàtica de màquines de recompensa (*Reward Machines*, RM) en l'aprenentatge per reforç s'ha explorat recentment amb grans models fundacionals, que generen RM directament a partir de descripcions de tasques en llenguatge natural. Tot i ser prometedors, aquests mètodes no ofereixen garanties formals i depenen d'avaluacions qualitatives, fetes per persones, de la correcció de les RM. En aquest treball proposem FARM (*A Formal Approach to Reward Machine Generation in Reinforcement Learning*), una cadena de processament basada en la lògica temporal lineal (LTL) per a generar RM: les tasques en llenguatge natural es tradueixen primer a fórmules LTL, que després es compilen en màquines de recompensa amb eines de la teoria d'autòmats. Aquest enfocament produeix RM que, per construcció, estan alineades amb l'especificació temporal desitjada. Introduïm un conjunt de mètriques estructurals i de comportament per a avaluar la qualitat de les RM, i avaluem el rendiment de l'aprenentatge per reforç en termes de taxa d'èxit i d'eficiència mostral en la resolució de tasques. En un banc de proves de tasques amb estructura temporal, el nostre mètode basat en LTL produeix RM més fidels a l'especificació de la tasca que les RM generades per un enfocament de l'estat de l'art basat en models fundacionals.

Evidence pending. The learning runs of Chapter 7: success rate and sample efficiency against the rebuilt ARM-FM.

If the evidence supports it: Els agents entrenats amb aquestes RM aprenen de manera més ràpida i fiable. Aquests resultats suggereixen que la síntesi formal de RM guiada per la lògica ofereix una alternativa robusta i interpretable als mètodes de generació de recompenses purament neuronals.

If it does not: Els agents entrenats amb aquestes RM, però, no aprenen més ràpid. Aquests resultats suggereixen que la síntesi formal de RM guiada per la lògica ofereix una alternativa interpretable i verificable als mètodes de generació de recompenses purament neuronals, amb el guany en les màquines i no en la velocitat d'aprenentatge.

Paraules clau: Aprenentatge per reforç; Màquina de recompensa; Lògica temporal lineal; Especificació de tasques en llenguatge natural

Resumen

La construcción automática de máquinas de recompensa (*Reward Machines*, RM) en el aprendizaje por refuerzo se ha explorado recientemente con grandes modelos fundacionales, que generan RM directamente a partir de descripciones de tareas en lenguaje natural. Aunque prometedores, estos métodos carecen de garantías formales y dependen de evaluaciones cualitativas, realizadas por personas, de la corrección de las RM. En este trabajo proponemos FARM (*A Formal Approach to Reward Machine Generation in Reinforcement Learning*), una cadena de procesamiento basada en la lógica temporal lineal (LTL) para generar RM: las tareas en lenguaje natural se traducen primero a fórmulas LTL, que después se compilan en máquinas de recompensa con herramientas de la teoría de autómatas. Este enfoque produce RM que, por construcción, están alineadas con la especificación temporal deseada. Introducimos un conjunto de métricas estructurales y de comportamiento para evaluar la calidad de las RM, y evaluamos el rendimiento del aprendizaje por refuerzo en términos de tasa de éxito y eficiencia muestral en la resolución de tareas. En un banco de pruebas de tareas con estructura temporal, nuestro método basado en LTL produce RM más fieles a la especificación de la tarea que las RM generadas por un enfoque del estado del arte basado en modelos fundacionales.

Evidence pending. The learning runs of Chapter 7: success rate and sample efficiency against the rebuilt ARM-FM.

If the evidence supports it: Los agentes entrenados con estas RM aprenden de forma más rápida y fiable. Estos resultados sugieren que la síntesis formal de RM guiada por la lógica ofrece una alternativa robusta e interpretable a los métodos de generación de recompensas puramente neuronales.

If it does not: Los agentes entrenados con estas RM, sin embargo, no aprenden más rápido. Estos resultados sugieren que la síntesis formal de RM guiada por la lógica ofrece una alternativa interpretable y verificable a los métodos de generación de recompensas puramente neuronales, con la ganancia en las máquinas y no en la velocidad de aprendizaje.

Palabras clave: Aprendizaje por refuerzo; Máquina de recompensa; Lógica temporal lineal; Especificación de tareas en lenguaje natural

Abstract

The automated construction of Reward Machines (RMs) in Reinforcement Learning has recently been explored using large foundation models, which generate RMs directly from natural language task descriptions. While promising, these methods lack formal guarantees and rely on qualitative, human-evaluated assessments of RM correctness. In this work, we propose FARM: A Formal Approach to Reward Machine Generation in Reinforcement Learning. A pipeline based on Linear Temporal Logic (LTL) for RM generation: natural language tasks are first translated into LTL formulas, which are then compiled into Reward Machines using automata-theoretic tools. This approach produces RMs that, by construction, are aligned with the desired temporal specification. We introduce a set of structural and behavioral metrics to evaluate RM quality, and assess reinforcement learning performance in terms of success rate and sample efficiency in task resolution. On a benchmark of temporally structured tasks, our LTL-based method produces RMs more faithful to the task specification than RMs generated by a state-of-the-art foundation model approach.

Evidence pending. The learning runs of Chapter 7: success rate and sample efficiency against the rebuilt ARM-FM.

If the evidence supports it: Agents trained with these RMs learn faster and more reliably. These results suggest that logic-guided formal RM synthesis offers a robust and interpretable alternative to purely neural reward generation methods.

If it does not: Agents trained with these RMs, however, do not learn faster. These results suggest that logic-guided formal RM synthesis offers an interpretable and verifiable alternative to purely neural reward generation methods; its gain lies in the machines, not in learning speed.

Key words: Reinforcement Learning; Reward Machine; Linear Temporal Logic; Natural Language Task Specification

Contents

Contents vii
List of Figures xi
List of Tables xii

Acknowledgements xiii
Glossary xv
1 Introduction 1
1.1 Motivation . . . 1
1.2 Objectives . . . 1
1.3 Expected impact . . . 1
1.4 Structure of the paper . . . 1
1.5 Scope and limits of the study . . . 1
2 State of the art 3
2.1 Related work . . . 3
2.1.1 Reward Machines . . . 3
2.1.2 ARM-FM . . . 4
2.1.3 Linear Temporal Logic . . . 5
2.1.4 Hard and soft constraints in planning . . . 6
2.1.5 Conclusions . . . 6
2.2 Critique of the state of the art . . . 6
2.2.1 Narrow task variety . . . 6
2.2.2 Inconsistent and unstructured generation . . . 6
2.2.3 Verification by a human or another model . . . 7
2.2.4 Reproducibility. . . . 7
3 Proposal 9
3.1 Human preferences . . . 9
3.2 No human expert . . . 9
3.3 Formal and robust generation: Schemas . . . 9
4 FARM: A Formal Approach to Reward Machine Generation 11
4.1 Introduction and motivation . . . 11
4.2 Global overview of the pipeline . . . 11
4.2.1 Running examples . . . 13
4.3 Step 1 · A language model chooses template clauses . . . 13
4.3.1 IBM nl2ltl . . . 13
4.3.2 Templates . . . 13
4.3.3 Two kinds of templates: goals and rules . . . 14
4.3.4 Adaptation to Reward Machine generation (prompt) . . . 14
4.3.5 Hard and soft clauses . . . 14
4.3.6 Task schemas . . . 14
4.3.7 Running examples: from task to formulas . . . 15
4.4 Step 2 · MONA turns each formula into a DFA . . . 15
4.4.1 FL-AT and MONA . . . 15

4.4.2 Redundant states in MONA's automata . . . 16
4.4.3 Running examples: from formulas to DFAs . . . 16
4.5 Step 3 · FL-AT multiplies the DFAs into one machine . . . 17
4.5.1 The product of the clause DFAs . . . 17
4.5.2 What was wrong with FL-AT's reward . . . 17
4.5.3 Final states . . . 18
4.5.4 Running examples: from DFAs to the product . . . 18
4.6 Step 4 · FARM's post-process: hard and soft clauses, and rewards . . . 19
4.6.1 Merging MONA's start state . . . 20
4.6.2 Identifying final states . . . 20
4.6.3 Sending failed states to a rejecting sink . . . 20
4.6.4 Reward assignment . . . 20
4.6.5 Recurrent soft-rule templates . . . 21
4.6.6 Rule-only machines . . . 22
4.6.7 Open items at the end of an episode . . . 22
4.6.8 Breaking a rule never pays better than running out of time . . . 22
4.6.9 The rewards on one task . . . 23
4.6.10 Running examples: from the product to the Reward Machine . . . 24
4.7 Step 5 · The critic: review, retries and thinking level . . . 26
4.7.1 A review step for the final version . . . 26
4.7.2 LLMs can hallucinate . . . 26
4.7.3 Different LLMs for generation and critique . . . 27
4.7.4 Number of iterations . . . 27
4.7.5 Fixing mistakes . . . 27
4.7.6 Flagging clauses whose states cannot occur in the environment . . . 27
4.7.7 Thinking level of the models . . . 27
4.7.8 Running examples: the verdict of the critic . . . 27
4.8 The output format . . . 27
4.9 Details . . . 28
4.9.1 Contributions . . . 28

5 Limitations **31**
5.1 The task is assumed to be achievable . . . 31
5.2 Contradictions . . . 31
5.3 Unknown facts . . . 31
5.4 Shortest path . . . 31
5.5 Human language can be misleading . . . 31

6 Evaluation methodology **33**
6.1 Empirical evaluation in a real environment . . . 33
6.2 Qualitative evaluation of the Reward Machines . . . 33
6.2.1 From traces to rewards . . . 33
6.2.2 Shape, number of states and useless states . . . 33

7 Results **35**
7.1 Comparison between ARM-FM and FARM . . . 35
7.1.1 Learning with the machines . . . 35
7.2 Reacting to events . . . 35

8 Conclusions **37**

9 Future work **39**

Bibliography **41**

Appendices

A Sustainable Development Goals **43**

B AI usage **45**

C Development tools **47**

D Implementation details **49**
D.1 IBM nl2ltl 49
D.2 FL-AT 49
D.3 Models and settings 49
D.4 How the examples were produced 50

E IBM template examples **51**

F MONA automata before and after cleaning **53**

G Running examples **55**
G.1 Example 2: a hard rule 55
G.2 Example 3: a soft preference 56
G.3 Example 4: a task made only of rules 56

H The to-do list on more tasks **59**
H.1 An alarm and a power cut 59
H.2 A box that falls off the shelf, and a key 61
H.3 A door and a box 63

I Formatting examples **65**

List of Figures

4.1 The FARM pipeline 12
4.2 Step 1 of FARM 13
4.3 Step 2 of FARM 15
4.4 DFAs of the running examples 17
4.5 Step 3 of FARM 17
4.6 Product of running example 3 19
4.7 Product of running example 2 19
4.8 Step 4 of FARM 20
4.9 Reward Machine of running example 3 25
4.10 Reward Machine of running example 2 25
4.11 Reward Machine of running example 1 26
4.12 Step 5 of FARM 26

F.1 The automaton of Absence(`key_lost`) before and after the two removals of redundant states. 54

G.1 The graph of the Reward Machine of example 4. 57

I.1 Example of a single image. 65
I.2 Example of two subfigures side by side. 65
I.3 Example of a TikZ diagram. 66
I.4 Example of a two-panel plot with illustrative data 66

List of Tables

4.1 The five steps of FARM. 12
4.2 The eight Declare templates of IBM nl2ltl with their LTLf definition and kind. 14
4.3 Clauses and LTLf formulas of the running examples 15
4.4 Product states of running examples 3 and 2 18
4.5 Rewards of the Reward Machines that FARM builds. 21
4.6 The to-do list of each state of example 2. The score is Φ, and the charge is what the machine adds when the environment ends the episode in the state. 23
4.7 Rewards of six traces of example 2, step by step. Idle steps pay nothing and are left out. When the environment ends the episode, the last reward includes the charge for the items still open. 24

H.1 The to-do list of each state of the alarm machine. 59
H.2 Steps of the alarm machine in which several events happen at once. 60
H.3 The to-do list of each state of the box and key machine. 61
H.4 The edges of the box and key machine that pay a reward. 62
H.5 Rewards of six traces of the door and box task, step by step. Idle steps are left out; when the environment ends the episode, the last reward includes the charge for the items still open. 63

I.1 Example of a `booktabs` table with illustrative data. 66

Acknowledgements

Glossary

CHAPTER 1
Introduction

1.1 Motivation

1.2 Objectives

1.3 Expected impact

1.4 Structure of the paper

1.5 Scope and limits of the study

CHAPTER 2
State of the art

2.1 Related work

2.1.1. Reward Machines

In reinforcement learning (RL), an agent learns to act in a Markov decision process (MDP) by maximising the reward it receives [1]. Most methods treat the reward function as a black box: the agent observes the number returned after each step and nothing about how it is computed [2]. In practice a person writes that function by hand, so its structure is known and can be exploited to learn faster.

Sparse rewards pose a second problem. When the reward arrives only on completing a long task, the agent receives almost no signal while it learns. Reward shaping adds intermediate reward to guide it. Ng et al. [3] prove that shaping with the difference of a potential function over states, $\gamma\Phi(s') - \Phi(s)$, leaves the optimal policy unchanged, and that no other form of shaping gives this guarantee in general.

Reward Machines (RMs) address both problems by exposing the structure of the reward to the learner. Toro Icarte et al. [4] introduced them at ICML 2018 and gave a fuller account in the Journal of Artificial Intelligence Research (JAIR) [2]. This thesis follows the JAIR article for the formalisation, counterfactual learning and reward shaping.

An RM is a finite-state machine, written $\langle U, u_0, F, \delta_u, \delta_r \rangle$ in the JAIR formulation. U is a finite set of states, $u_0 \in U$ the initial state and F the set of terminal states. The transition function δ_u takes the current state and the set of propositions that hold and returns the next state. The reward function δ_r assigns to each state a reward function over environment transitions. The ICML formulation **attaches the reward to the pair of states of a transition instead**, which is the convention of the RM text format used in this thesis.

Propositions come from outside the machine. A labelling function L maps each environment transition to the set of propositions that hold in it, for example "the agent holds the key". At every step the machine reads that set and moves. The agent learns in the product of the MDP and the RM, whose states pair an environment state with an RM state; in the product the reward is Markovian even when the task is not.

Exposing the structure enables dedicated learning methods. Q-learning for RMs (QRM) [4] decomposes the machine and learns, off-policy, one sub-policy per RM state in parallel. The JAIR article adds counterfactual reasoning for RMs (CRM), which reuses each experience to update every RM state; a hierarchical method (HRM); and automated reward shaping. The shaping potential is the negative optimal value of the RM state,

$\Phi(s, u) = -v^*(u)$, computed by value iteration on the machine (Section 4.4 of the article, restricted to simple RMs).

RMs have two costs. Someone has to design the machine, and the authors note that RMs do not remove the difficulty of designing reward functions, although they may make temporally extended behaviours easier to specify [2]. The labelling function is also assumed to exist and to be correct: learning RMs from experience, one of the open problems listed in the article, needs a correct labelling function. Both costs motivate generating RMs automatically.

2.1.2. ARM-FM

ARM-FM (Automated Reward Machines via Foundation Models), by Creus Castanyer et al. [5] and published at ICLR 2026, replaces the hand-built RM with one generated by a foundation model (FM) from a natural-language task description. The output is a Language-Aligned Reward Machine (LARM), made of three parts: an RM, the Python code of its labelling functions, and a natural-language description of each RM state. A text embedding of the description of the current state is given to the policy together with the environment observation, so that the policy knows which sub-goal is active. In this thesis **ARM-FM is the method and LARM its output**.

The RM is produced by a generator model and reviewed by a critic model over several rounds of self-improvement. A human may optionally approve the result or give corrective feedback. The generator prompt asks for the machine as plain text in a fixed format. The critic prompt asks, among other checks, that regressions are penalised and that no unintentional loops exist, and the critic answers either `NO CHANGES NEEDED` or `CHANGES REQUIRED` [5, pp. 26–27]. The paper uses GPT-4o to generate all LARM components, except for the 1 000 XLand-MiniGrid LARMs, which come from several open-source models.

The evaluation covers four environments: MiniGrid (DoorKey and the harder UnlockToUnlock, BlockedUnlockPickup and KeyCorridor), Craftium (a Minecraft-based environment where the agent mines a diamond after gathering wood, stone and iron), Meta-World manipulation tasks, and XLand-MiniGrid. On DoorKey the agent trained with the LARM outperforms DQN with ICM, ReAct and plain DQN; on the three harder MiniGrid tasks it is the only agent that solves all of them; on Craftium it learns where an agent without the LARM makes no progress. On XLand-MiniGrid a single policy trains on ten tasks at once and solves unseen combinations of known tasks without fine-tuning.

Listing 2.1 reproduces the DoorKey machine printed in the appendix of the paper.

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3
INITIAL_STATE: u0
TRANSITION_FUNCTION:
(u0, has_key) -> u1
(u0, else) -> u0
(u1, is_door_in_env_open) -> u2
(u1, not_has_key) -> u0
(u1, else) -> u1
(u2, at_goal) -> u3
(u2, else) -> u2
(u3, else) -> u3
REWARD_FUNCTION:
(u0, has_key, u1) -> 0.2
(u1, is_door_in_env_open, u2) -> 0.3
(u1, not_has_key, u0) -> -0.2
(u2, at_goal, u3) -> 1.0
```

Listing 2.1: The DoorKey Reward Machine printed by ARM-FM (page 30 of the paper).

FARM keeps this text format for its machines and extends it (Section 4.8).

2.1.3. Linear Temporal Logic

Linear Temporal Logic (LTL) is a logic for stating properties of sequences of states, called traces. Pnueli [6] proposed it to reason about programs. It is a language rather than a catalogue of patterns: a formula is built from propositions, Boolean connectives and four temporal operators.

- X φ (next): φ holds at the next step.
- F φ (eventually): φ holds at the current step or at a later one.
- G φ (always): φ holds at every step from the current one on.
- φ U ψ (until): ψ holds at some step from the current one on, and φ holds at every step before it.

LTL is defined over infinite traces, whereas a reinforcement learning episode ends. LTLf, LTL over finite traces [7], keeps the syntax and reads a formula over a trace of finite length, where next requires a following step to exist. LTLf is as expressive as first-order logic over finite traces and corresponds to the star-free regular expressions.

In the DoorKey task of Chapter 4, the propositions are `has_key`, `door_open` and `at_goal`. "The agent reaches the goal at some point" is

$$\mathrm{F}\ \texttt{at_goal},$$

and "the door is not open before the key is held" is

$$(\neg\ \texttt{door_open}\ \mathrm{U}\ \texttt{has_key}) \vee \mathrm{G}\ \neg\ \texttt{door_open}.$$

For every LTLf formula there is a deterministic finite automaton (DFA) that accepts exactly the traces satisfying it [7]. In practice the formula is rewritten in monadic second-order logic over finite words, from which MONA [8] builds the minimal DFA. This is the bridge used by Step 2 of FARM (Section 4.4).

Camacho et al. [9] bring this logic to RL. They show that RMs can express reward specifications written in LTL, LTLf and other formal languages, translate a specification into an RM and combine several specifications with a product construction.

Declare [10] defines a catalogue of templates, such as Existence, Response or Precedence, each with an LTL semantics, to specify business processes declaratively. nl2ltl [11] translates natural language into LTL by letting a language model choose a Declare template and its arguments.

2.1.4. Hard and soft constraints in planning

In automated planning, a plan is a sequence of actions that takes a system from an initial state to a goal, and PDDL is the standard language to describe such problems. PDDL3 [12] adds two things to it. State trajectory constraints are conditions on the whole sequence of states that the plan goes through, not only on the last one, written with operators such as `always`, `sometime`, `at-most-once` and `sometime-before`, close to those of LTL. Preferences are soft constraints: a valid plan must satisfy every hard constraint, while each preference it breaks only adds a cost to the plan. Goals and trajectory constraints can both be stated as preferences, so a condition over time can be either required or only preferred.

2.1.5. Conclusions

Each line of work covers part of the problem. Reward Machines expose the structure of a reward but are built by hand. ARM-FM generates them from language, without formal guarantees on the result. LTLf gives task statements a precise meaning and a route to automata, and planning separates the rules that must hold from the preferences that only cost. Chapter 3 combines these pieces.

2.2 Critique of the state of the art

2.2.1. Narrow task variety

The tasks whose Reward Machines ARM-FM prints are all simple sequences of steps: first one thing, then the next, then the goal. DoorKey goes `key`, `door`, `goal`; UnlockToUnlock repeats `key` and `door` before the `goal`; Craftium gathers `wood`, `stone`, `iron` and `diamond` in order; Meta-World grasps, moves and succeeds [5, pp. 30–32]. Each machine is a chain of states, one per step. No task states a rule that must hold throughout the episode, such as keeping the key until the door is open, and none separates what must happen from what is only preferable. The closest the machines come to a rule is a penalty for losing a key, for example -0.1 in UnlockToUnlock.

2.2.2. Inconsistent and unstructured generation

Language models do not return the same output on every run, even with the temperature set to zero. Ouyang et al. [13] measured this for ChatGPT on 829 code-generation problems, and the same task can therefore yield different machines. ARM-FM places no schema on what the model writes: the machine is plain text in a format described by the prompt, and the rewards are free numbers.

Free rewards allow loops that pay without progress. Proposition 1 of ARM-FM guarantees that optimising the LARM also solves the sparse task, provided the machine has no positive reward cycle [5, p. 21]. The paper describes no automatic check of this condition; the critic prompt only asks for penalised regressions and no unintentional loops [5, p. 27]. The Meta-World machine printed in the paper breaks the condition: the cycle $u_0 \to u_1 \to u_2 \to u_0$ pays $+0.20$, $+0.40$ and -0.40, a total of $+0.20$ per lap [5, p. 32], although the paper states that its LARMs satisfy the property. The DoorKey loop that pays $+0.2$ for picking the key up and -0.2 for dropping it sums to zero, which satisfies the condition only because two chosen values cancel.

A critic model can hallucinate as well [14], so a critic, even with a human at its side, is not a guarantee.

2.2.3. Verification by a human or another model

ARM-FM states the limitation itself: human verification during RM generation is a trade-off that presupposes access to human verifiers, and reducing it by exploiting the automaton structure, for example through formal verification, is left as future work [5, pp. 9–10]. Humans took part in four of the seven settings of its Table 3. In DoorKey a human check confirmed the machine after three iterations; in UnlockPickup a human added the transition for a dropped key; in KeyCorridor a human suggested intermediate rewards; and in Meta-World a human rescaled the rewards. BlockedUnlockPickup and Craftium needed no intervention, and the 1 000 XLand-MiniGrid machines were scored by a second language model instead.

2.2.4. Reproducibility.

The ARM-FM paper releases no code. Its appendices print the generator and critic prompts, six Reward Machines, the labelling functions of four MiniGrid tasks and the hyperparameters. A public repository of the first author [15], which the paper does not reference, adds prompts, generated machines and labelling code for four MiniGrid tasks only. This material allows the method to be rebuilt but not its exact outputs, so the ARM-FM baseline of this thesis is a reconstruction whose results may differ from the original ones. The authors did not answer the contact attempts made for further material.

CHAPTER 3
Proposal

3.1 Human preferences

An RL agent pursues whatever its reward pays for, so the reward is the way a person tells the agent what to achieve [1]. AI alignment aims to make AI systems behave in line with human intentions and values [16]. For an RL agent, one part of this aim is a reward that says everything the person means: the goal, the order of the steps, the rules that must not be broken and the preferences to keep when possible.

A task is rarely a single chain of steps. FARM lets the user state four things in natural language: what to do, in which order, what to do in response to an event, and what must never happen. The temporal relations of order, response and prohibition are the vocabulary of the LTLf templates of Section 2.1.3.

Each statement is either hard or soft. A hard statement must hold for the task to count as done. A soft statement is a preference: the agent may give it up, at a cost. This split between constraints and preferences comes from planning (Section 2.1.4) and is carried here into Reward Machines.

3.2 No human expert

ARM-FM relies on a human or on a second model to verify its machines (Section 2.2). FARM aims at generation without a human expert. The language model only chooses clauses from a fixed catalogue, and a compiler builds the machine from them, so the properties of the machine follow from the construction and do not depend on a reviewer. A critic model still reads the result, but it can only flag clauses; the generator retunes them, and the critic never edits the machine.

3.3 Formal and robust generation: Schemas

FARM limits the role of the language model. Its answer is bounded: instead of writing a Reward Machine, the model only picks patterns from a fixed catalogue, the Declare templates of Section 2.1.3, and fills them with the propositions of the environment. Any other answer is rejected. Everything after that choice is formal and deterministic. The machine is built from the chosen patterns alone, without the meaning of the propositions, so tasks of the same shape, such as any "first a, then b, then c", receive the same machine with only the propositions exchanged. These task schemas (Section 4.3.6) make the result predictable and confine the variability of the language model to that one choice. The

rewards are designed so that the agent cannot earn reward by going round in circles, and every machine is checked for such loops.

CHAPTER 4

FARM: A Formal Approach to Reward Machine Generation

4.1 Introduction and motivation

4.2 Global overview of the pipeline

FARM turns a natural-language task into a Reward Machine in five steps. A language model chooses clauses, fixed tools compile them into automata, a post-process asigns rewards and tweaks the result and a critic reviews it. Only first and last steps are not formal: the language model is the only source of variability in the system. Figure 4.1 draws the pipeline, and Table 4.1 gives the input, output and tool of each step.

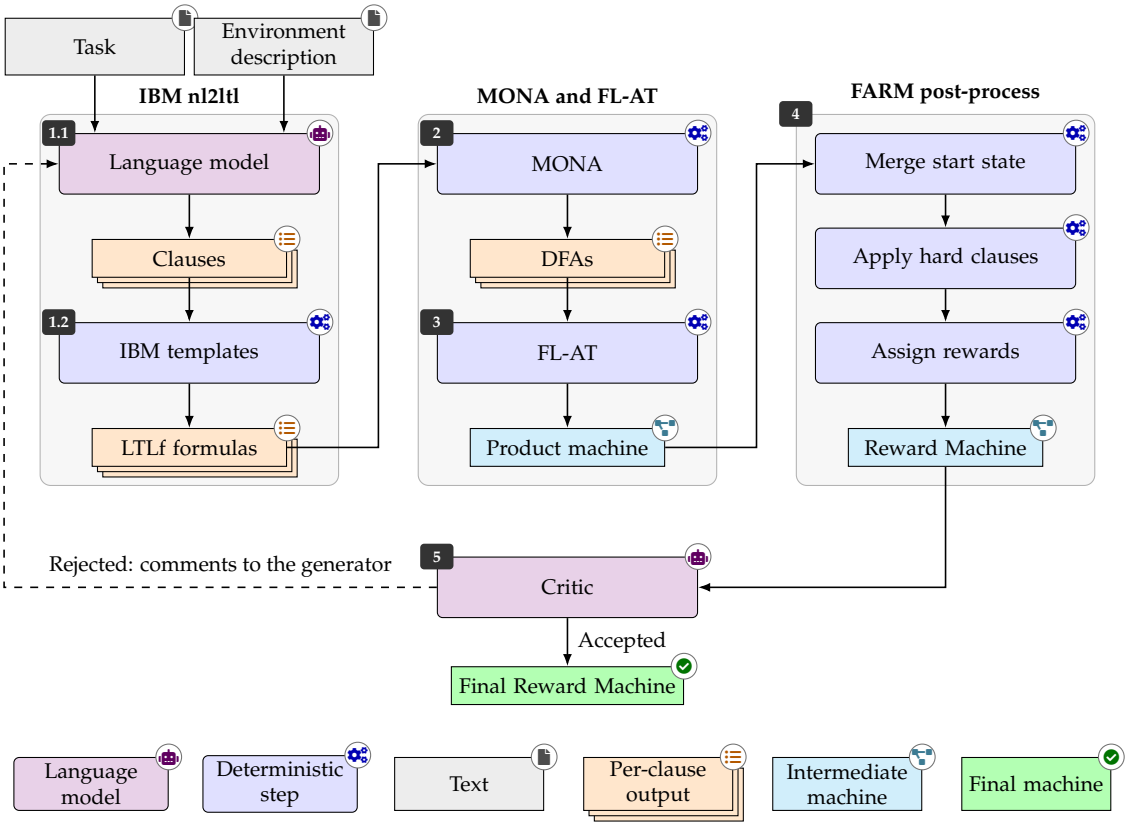


Figure 4.1: The FARM pipeline. Rounded boxes are processes and square boxes are data. The colour and the corner icon give the kind of each box, as in the legend, and are the same in every diagram of this chapter. A stack stands for one item per clause.

Table 4.1: The five steps of FARM.

Step	Input	Output	Tool	Description
1.1	Task and environment description	Clauses: template, propositions, hard or soft	Language model	The model chooses clauses from eight Declare templates.
1.2	Clauses	One LTLf formula per clause	IBM nl2ltl templates	The template of each clause writes its LTLf formula.
2	LTLf formulas	One DFA per formula	FL-AT; MONA	FL-AT rewrites each formula for MONA, and MONA builds its DFA.
3	One DFA per formula	Product Reward Machine and clause states	FL-AT	The product follows all clauses at once and records where each one stands.
4	Product Reward Machine and clause states	Final Reward Machine	FARM post-process	Redundant states are merged, hard clauses set the final states and the rejecting well, soft clauses cost reward, and new rewards replace those of FL-AT.
5	Task, clauses and Reward Machine	Verdict and comments on clauses	Critic model	The critic accepts the result or flags clauses, which the generator retunes.

4.2.1. Running examples

Three tasks in the DoorKey environment of ARM-FM run through the chapter. The environment provides four propositions: `has_key`, `door_open`, `at_goal` and `key_lost`, the event of dropping the key while the door is still closed.

1. A sequence: *Pick up the key, then open the door, then reach the goal.*
2. A hard rule: *Reach the goal, and if you drop the key, pick it up right away.*
3. A soft preference: *Reach the goal, ideally without ever losing the key.*

Each step ends by applying itself to the three tasks, and its output is the input of the next step: Step 1 gives the clauses and their formulas, Step 2 the automata, Step 3 their product, Step 4 the Reward Machine and Step 5 the verdict of the critic.

4.3 Step 1 · A language model chooses template clauses

The first step reads the task and the description of the environment and returns one LTLf formula per clause (Figure 4.2).

A language model first chooses the clauses. A clause names one of eight templates, the propositions it applies to and a priority, hard or soft. The output is restricted to this form, so nothing else reaches the compiler. The template of each clause then writes its LTLf formula.

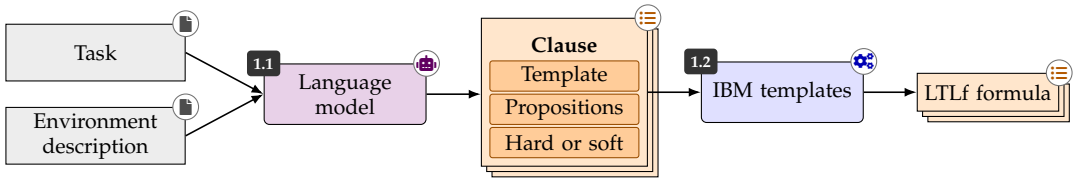


Figure 4.2: Step 1. The language model chooses the clauses, each a template with its propositions and a priority, and the template of each clause writes its LTLf formula. A stack stands for one item per clause.

4.3.1. IBM nl2ltl

IBM's nl2ltl [11] converts a sentence into an LTLf formula. It keeps one class per Declare template [10]; each class holds the LTLf definition of its template and builds the formula from the propositions given to it. Its default engine asks a GPT model for the template and its arguments. FARM replaces that engine and keeps the template classes (Appendix D).

4.3.2. Templates

Table 4.2 lists the eight templates with their LTLf definitions in IBM's library.

Table 4.2: The eight Declare templates of IBM nl2ltl with their LTLf definition and kind.

Template	LTLf	Reads as	Kind
Existence(a)	$\mathrm{F}\,a$	a happens at some point	goal
ExistenceTwo(a)	$\mathrm{F}\,(a \wedge \mathrm{X}\,\mathrm{F}\,a)$	a happens at least twice	goal
Absence(a)	$\neg\,\mathrm{F}\,a$	a never happens	rule
RespondedExistence(a, b)	$\mathrm{F}\,a \rightarrow \mathrm{F}\,b$	if a ever happens, b happens too	rule
Response(a, b)	$\mathrm{G}\,(a \rightarrow \mathrm{F}\,b)$	every a is answered by b at the same step or later	rule
Precedence(a, b)	$(\neg b\,\mathrm{U}\,a) \vee \mathrm{G}\,\neg b$	b cannot happen before a	rule
ChainResponse(a, b)	$\mathrm{G}\,(a \rightarrow \mathrm{X}\,b)$	every a is followed by b at the next step	rule
NotCoExistence(a, b)	$\mathrm{F}\,a \rightarrow \neg\,\mathrm{F}\,b$	if a ever happens, b never does	rule

4.3.3. Two kinds of templates: goals and rules

A clause is a goal if a step in which no proposition holds leaves it unsatisfied, and a rule if that step leaves it satisfied. Existence and ExistenceTwo are goals: they ask for something to happen. The other six templates are rules, Precedence included: they hold while nothing happens, and a later event may break them. The compiler applies the test to the DFA of each clause, so no list of templates is needed.

4.3.4. Adaptation to Reward Machine generation (prompt)

The default prompt of IBM's engine is a few-shot list that allows five of the eight templates, with examples about business applications such as Slack or Salesforce, no definitions and no description of the environment. The prompt of FARM covers all eight templates. It gives the model the catalogue with a definition of each template, a table from typical phrases to templates, worked examples, the rules for choosing hard or soft, the environment description in Markdown, the task and, on a retry, the comments of the critic. The propositions the model may use come from the environment description, so the same prompt serves any environment.

4.3.5. Hard and soft clauses

Each clause carries a priority that the model chooses for that clause alone. The model marks a clause soft only when the task uses explicit preference language such as "if possible" or "ideally"; every other clause is hard. The priority never changes the formula. A hard clause must hold for the task to be completed. A soft clause is a preference: breaking it costs reward but does not prevent finishing (Step 4). IBM's templates have no such notion, since "never touch the lava" and "if possible, never touch the lava" are the same Absence clause, so the priority has to come from the prompt.

4.3.6. Task schemas

Steps 2 to 4 use only the template and the priority of each clause and the place each proposition fills, never what a proposition means. Tasks with the same clauses up to the propositions therefore receive the same machine with the names exchanged, and the clauses with the propositions left open form a task schema. Each running example stands for its schema: a task such as "gather wood, then stone, then iron" would have the clauses

of example 1 with `has_wood`, `has_stone` and `has_iron`, and the same machine. The priority belongs to the schema, so example 3 with a hard Absence would give a different machine, with a rejecting well. Two tasks of the same shape share a machine only when the generator gives them the same clauses.

4.3.7. Running examples: from task to formulas

Table 4.3 gives the clauses that the language model chose for each running example and the LTLf formula that the template of each clause wrote. The sequence of example 1 becomes three goals, one Existence for each step, and two Precedence rules that fix their order. Example 2 joins the goal with a ChainResponse rule, the template for "right away". In example 3 the word "ideally" makes the Absence of a lost key soft, and its formula is the same as that of a hard Absence.

Table 4.3: The clauses and LTLf formulas of the running examples after Step 1. All clauses are hard unless marked soft.

#	Clauses	LTLf
1	*Pick up the key, then open the door, then reach the goal*	
	Existence(`has_key`)	$\mathrm{F}\,\texttt{has_key}$
	Precedence(`has_key`, `door_open`)	$(\neg\texttt{door_open}\ \mathrm{U}\ \texttt{has_key}) \vee \mathrm{G}\,\neg\texttt{door_open}$
	Existence(`door_open`)	$\mathrm{F}\,\texttt{door_open}$
	Precedence(`door_open`, `at_goal`)	$(\neg\texttt{at_goal}\ \mathrm{U}\ \texttt{door_open}) \vee \mathrm{G}\,\neg\texttt{at_goal}$
	Existence(`at_goal`)	$\mathrm{F}\,\texttt{at_goal}$
2	*Reach the goal, and if you drop the key, pick it up right away*	
	Existence(`at_goal`)	$\mathrm{F}\,\texttt{at_goal}$
	ChainResponse(`key_lost`, `has_key`)	$\mathrm{G}\,(\texttt{key_lost} \rightarrow \mathrm{X}\,\texttt{has_key})$
3	*Reach the goal, ideally without ever losing the key*	
	Existence(`at_goal`)	$\mathrm{F}\,\texttt{at_goal}$
	Absence(`key_lost`), soft	$\neg\,\mathrm{F}\,\texttt{key_lost}$

4.4 Step 2 · MONA turns each formula into a DFA

The second step turns every LTLf formula into a deterministic finite automaton (DFA) that accepts exactly the traces satisfying it (Section 2.1.3). FL-AT translates the formula for MONA, and MONA returns the automaton (Figure 4.3).

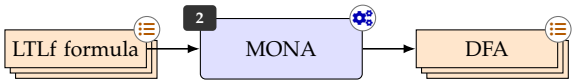


Figure 4.3: Step 2. Each LTLf formula becomes one DFA.

4.4.1. FL-AT and MONA

FL-AT, the Formal Language–Automaton Transmogrifier [17], writes the LTLf formula as a formula of monadic second-order logic over finite words, the input language of MONA, and reads back the automaton that MONA prints. MONA [8] is a decision procedure for that logic: to decide a formula it builds the minimal DFA of its models. FARM uses a fork of FL-AT (Appendix D).

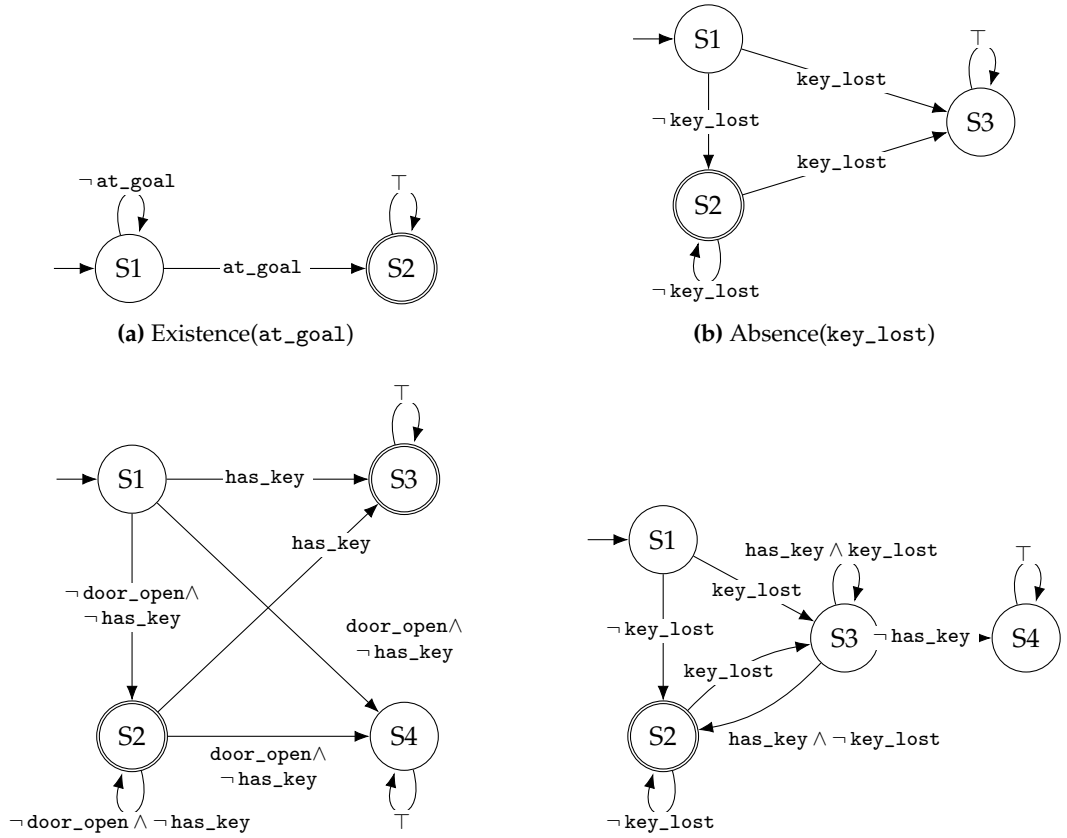
4.4.2. Redundant states in MONA's automata

The automaton that MONA returns is minimal for the language of its formula, yet two of its states are redundant for a Reward Machine. Figure F.1 shows both for the clause Absence(`key_lost`). The first is a start state that every input leaves for the same successor (S0 in panel (a)); the FL-AT fork removes it. The second appears in rule clauses: a "no step yet" start that nothing re-enters and that behaves like its idle successor on every input (S1 against S2 in panel (b)); the post-process of Step 4 merges the two (panel (c)).

4.4.3. Running examples: from formulas to DFAs

The three running examples have nine clauses but only four templates, and clauses of the same template give automata of the same shape. Figure 4.4 draws one automaton per template. All three examples share Existence(`at_goal`), Absence(`key_lost`) comes from example 3, Precedence(`has_key`, `door_open`) from example 1 and ChainResponse(`key_lost`, `has_key`) from example 2. The other Existence and Precedence clauses of example 1 differ only in their propositions. In these graphs an arrow without a source marks the initial state and a double circle an accepting state. Each edge is labelled with its guard, where $\neg$ stands for `!`, $\wedge$ for `&` and $\top$ for any input.

The Existence automaton waits in S1 until `at_goal` holds and then stays accepting in S2. Absence accepts in S2 while the key is kept and falls into S3, which no input leaves, at the first loss; it is the automaton of panel (b) of Figure F.1. Precedence accepts in S2 while neither event has happened and in S3 once the key comes first, and opening the door without the key leads to the dead state S4. ChainResponse accepts in S2 while nothing is owed. A lost key moves it to S3, where the key must be back at the next step, and without `has_key` it falls into S4. In the three rule automata S1 is the "no step yet" start of Section 4.4.2: it moves like S2 but is not accepting.



(a) Existence(`at_goal`) **(b)** Absence(`key_lost`)

(c) Precedence(`has_key`, `door_open`) **(d)** ChainResponse(`key_lost`, `has_key`)

Figure 4.4: The DFAs of the four templates of the running examples after Step 2, once the FL-AT fork has removed MONA's first start state.

4.5 Step 3 · FL-AT multiplies the DFAs into one machine

The third step combines the clause automata into one. FL-AT builds their synchronous product, the construction that Camacho et al. [9] use for combined specifications: a product state holds one state of each clause DFA, and every step moves all of them at once. The fork returns that tuple for every product state, so later steps know where each clause stands (Figure 4.5).

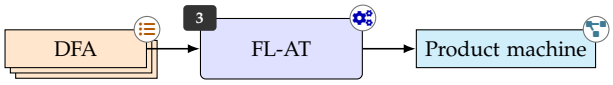


Figure 4.5: Step 3. The DFAs of all clauses become one product machine.

4.5.1. The product of the clause DFAs

The product has no more states than the product of the sizes of the clause DFAs [9]. Its transitions carry Boolean conditions over the propositions, one per combination of clause transitions that can fire together.

4.5.2. What was wrong with FL-AT's reward

FL-AT also assigns rewards. Each product edge pays the reward of every clause whose DFA is accepting after the step, so a finished clause is paid again at every later step.

Rules that merely hold pay for doing nothing, and Response and ChainResponse contain a rewarded cycle that can be farmed. FARM discards these rewards (Step 4).

4.5.3. Final states

FL-AT derives its accepting states from its own rewards: they are the destinations of the edges that pay the highest reward. FARM discards this marking and recomputes the final states from the hard clauses (Step 4).

4.5.4. Running examples: from DFAs to the product

FL-AT multiplies the automata of each example. The product of example 3 has five states (Figure 4.6) and that of example 2 seven (Figure 4.7). Example 1 has five clauses, and its product, with 14 states and 42 transitions, is too large to draw legibly. Product states are named p0, p1 and so on, and each edge is labelled *guard / reward* with the reward of FL-AT. Table 4.4 gives the clause states behind each product state of examples 3 and 2.

Table 4.4: The product states of examples 3 and 2 after Step 3. The clause states give the state of the Existence(`at_goal`) automaton first and that of the second clause after it. The reward is the one that FL-AT pays on every edge into the state.

Example	State	Clause states	Meaning	Reward
3	p0	(S1, S1)	Start	–
	p1	(S1, S2)	No goal yet, key kept	1
	p2	(S1, S3)	No goal yet, key lost	0
	p3	(S2, S2)	Goal, key kept	2
	p4	(S2, S3)	Goal, key lost	1
2	p0	(S1, S1)	Start	–
	p1	(S1, S2)	No goal yet, nothing owed	1
	p2	(S1, S3)	No goal yet, key owed	0
	p3	(S2, S2)	Goal, nothing owed	2
	p4	(S2, S3)	Goal, key owed	1
	p5	(S2, S4)	Goal, rule broken	1
	p6	(S1, S4)	No goal yet, rule broken	0

The reward of an edge counts the clause automata that accept after the step, so it depends only on the state the edge enters. In example 3 the agent earns 1 for every idle step in p1 before the goal and 2 for every step it stays in p3 after it. In example 2 it can lose the key after the goal and pick it up again: the cycle from p3 to p4 and back pays 1 and then 2, and nothing stops the agent from repeating it. The state p5, where the hard rule is already broken, still pays 1 per step, because the goal clause keeps accepting. In example 1 every edge pays between 2 and 5, so even reaching the goal first, before the key and the door, which breaks the task, pays 2. FL-AT marks as accepting the destinations of its highest-paying edges: p3 in examples 3 and 2, where both clauses accept, and the state of example 1 where all five do. In all three products p0 moves like p1 and neither is accepting, the trace of the “no step yet” starts of the rule automata.

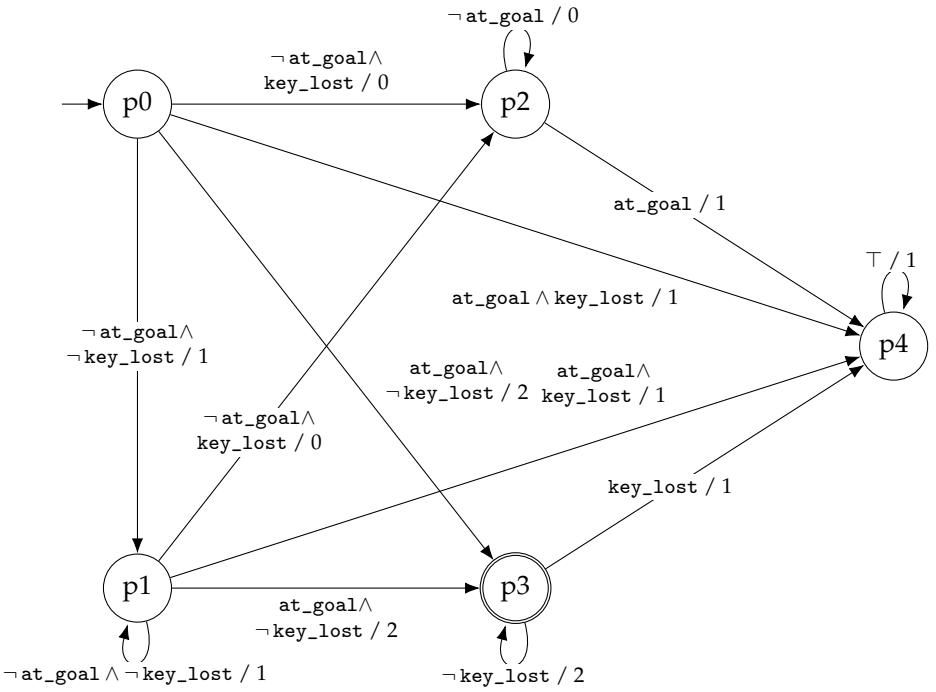


Figure 4.6: The product of example 3 after Step 3, with the rewards of FL-AT.

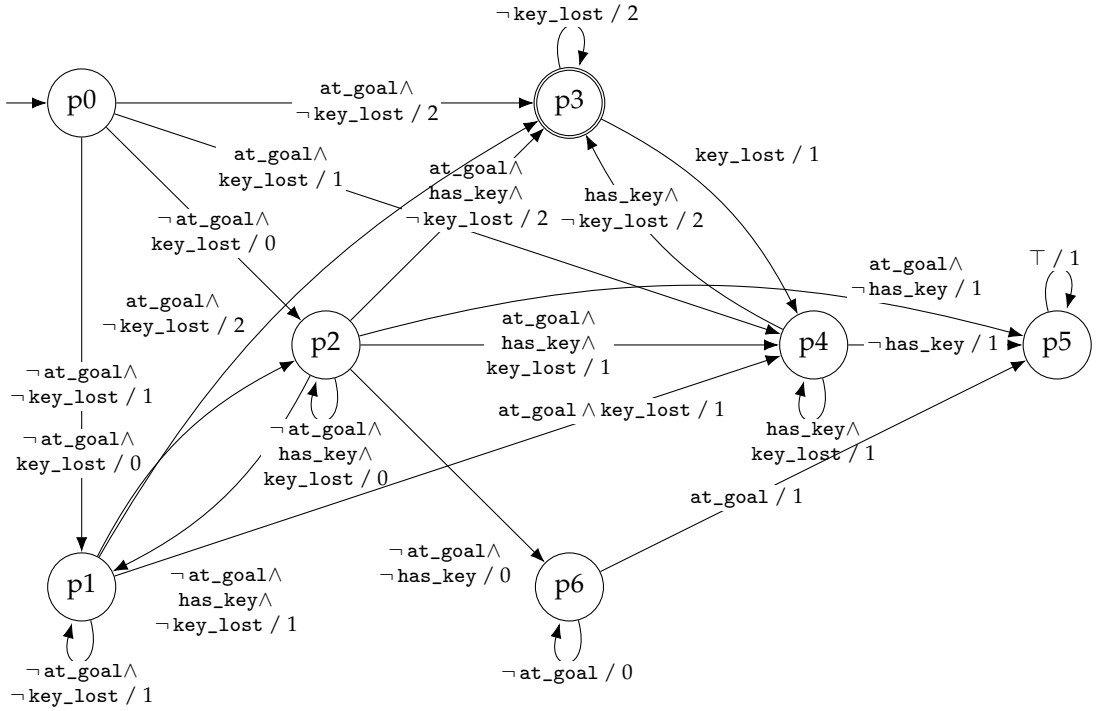


Figure 4.7: The product of example 2 after Step 3, with the rewards of FL-AT.

4.6 Step 4 · FARM's post-process: hard and soft clauses, and rewards

The fourth step is specific to FARM and turns the product into a Reward Machine. For every clause, each product state is labelled as satisfied, pending (not yet satisfied but still possible) or violated (no longer satisfiable). The labels are computed exactly, by reacha-

bility on the DFA of each clause. Hard clauses prune the machine and decide the final states, soft clauses only change rewards, and the rewards are assigned last (Figure 4.8).

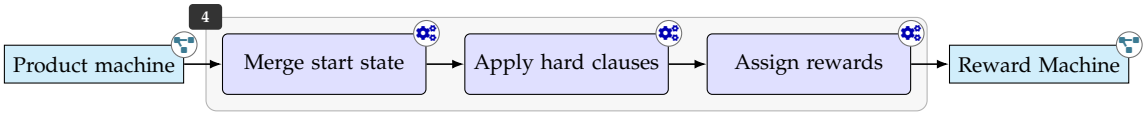


Figure 4.8: Step 4. FARM's post-process turns the product into a Reward Machine.

4.6.1. Merging MONA's start state

MONA gives each rule clause a "no step yet" start that is left on the first step and never re-entered. When that start goes to the same state as its idle successor on every input, the successor becomes the start and the old start is dropped. Otherwise the machine is unchanged. Without the merge, an idle first step would still pay progress.

4.6.2. Identifying final states

A final state is a state in which every hard clause is satisfied, and only a task with a hard goal has one. Soft clauses never affect the final condition: a preference broken on the way still allows the task to finish, at a cost.

4.6.3. Sending failed states to a rejecting sink

An edge into a state where a hard clause is violated goes to a single shared well, a rejecting state that ends the episode with a penalty. The penalty adds to the -1 of the break the charge for every item open in the state the edge leaves (Section 4.6.8), so the edges into the one well differ in their reward. In a task with a hard goal, the same happens to states from which no final state can be reached. A hard clause that is only pending, such as a Response still waiting for its answer, blocks finishing but does not end the episode.

4.6.4. Reward assignment

The rewards follow one rule for every task: getting closer to finishing a task or a rule pays, and stepping back from it or getting a new thing to do costs. The new thing to do can come from the agent, which broke a rule, or from the environment, which changed so that the instructions now ask for a reaction. FARM treats the task as a to-do list. Every clause, goal or rule, hard or soft, is an item that counts the steps it still needs. Each step done pays $+0.1$, and each step back or step of new work costs -0.1. When the episode ends, each item still open is charged, -1 if it is hard and -0.5 if it is soft, and a goal is a hard item. A few constants complete the rewards (Table 4.5).

Progress follows the potential-based form of Ng et al. [3]: a transition from u to u' adds

$$F(u, u') = \Phi(u') - \Phi(u), \qquad \Phi(u) = -0.1 \sum_i \ell_i(u_i),$$

where u_i is the state of clause i in the product state u, the tuple that the fork of FL-AT returns (Section 4.5), and $\ell_i(u_i)$ is the length of the longest path from u_i to an accepting state in the DFA of clause i. An accepting state needs no more steps, and the loops of each clause DFA collapse into one node, so every ℓ_i is finite. A soft clause broken for good counts zero, because it paid its penalty when it broke. Hard and soft items weigh the same, and $\Phi = 0$ at the final states. A step that advances an item pays 0.1 for each

step gained, staying pays nothing, and an event that gives an item more to do costs 0.1 for each step it adds. The longest path is used rather than the shortest because a DFA can have shortcut edges where two events happen in one step, which the environment may never produce, and these would make an item look closer to done than it is. Edges into the well, and into a state from which no target can be reached, pay no progress.

Measuring each item on its own DFA gives every event the same price wherever it happens. A potential measured on the whole product, as the longest path to a final state or to a state with nothing pending, has to collapse the loops of the product to stay finite, and a loop then pays nothing unless one of its ends is a target. Some rules would get no reward when they are triggered, although a clear reward exists. In example 2, dropping the key and picking it up at the next step moves the machine from u0 to u1 and back; the product-wide potential pays both edges 0, and the to-do list pays -0.1 for the drop and $+0.1$ for the pick-up. The product-wide potential also prices one event differently depending on what else is pending: in a rule-only task about an alarm, the same ring and switch-off paid $-0.3/+0.3$, $-0.1/+0.1$ or 0 depending on the state, and the to-do list pays $-0.1/+0.1$ in all three (Appendix H.1).

Table 4.5: Rewards of the Reward Machines that FARM builds.

Event	Reward
Progress, per step an item gains (undiscounted)	$+0.1$
More to do, per step an event adds to an item (undiscounted)	-0.1
Reaching the final state	$+1$
Entering the well (the episode ends)	-1
Breaking a soft rule	-0.5
A soft clause still pending on the finishing edge	-0.5
Episode end (environment or well), per open hard item, goals included	-1
Episode end (environment or well), per open soft item	-0.5

Progress is a difference of one score per state, so the differences cancel around any loop: a break that is repaired pays back exactly what it cost. The other rewards cannot repeat for gain: the finishing reward is paid only on edges into the final state, the penalty of the well only on edges into the well, both absorbing, a broken soft rule only costs, and the charge for open items is paid once, at the last step or on the edge into the well (Section 4.6.7). A Bellman–Ford check on the finished machine rejects any machine in which a loop still pays a positive total.

4.6.5. Recurrent soft-rule templates

An LTLf formula gives one verdict per trace, so a soft rule would cost one penalty per episode however often it is broken. For the four templates whose DFA has a violated state (Absence, Precedence, ChainResponse and NotCoExistence), FARM re-arms a soft clause: its violated state takes the edges of the start state, so every new break costs -0.5 again. A step that breaks the rule while it is still broken is free, and the rule is armed again on the next step. Existence and ExistenceTwo are excluded, because paying again would invite reward farming.

4.6.6. Rule-only machines

A task with no hard goal has no final state: only the well or the end set by the environment ends the episode, and the compiler warns the user that the task has no goal. The to-do list needs no special case for such a task. Many rules wait for an event. When the event happens, caused by the agent or by the environment, the rule adds an item, and the step costs -0.1 at once. The reaction that the rule asks for removes the item and pays the 0.1 back. In the rule "Whenever a window breaks, fix it", Response(`window_broken`, `window_fixed`), the window breaks while the agent does nothing, yet from that step the agent owes a repair. A window that breaks and is fixed totals 0, the same as a window that never breaks, whether the fix comes at the next step or three steps later. The penalty tells the agent that something needs a reaction, and the only reaction that earns it back is the one the user asked for. The same holds when the agent starts the wait itself: in example 4 (Appendix G.3), dropping the key costs -0.1 and picking it up again pays $+0.1$.

The graph gives a second reading. A state with an open item is a state to run away from: if the episode ends there, every open item is charged (Section 4.6.7), and the other ways out cost too: the well charges the broken hard rule and every item still open, and a broken soft rule costs -0.5. The only free way out is the edge that does what the rule or the preference asks, so running away from these states and doing the task are the same thing.

4.6.7. Open items at the end of an episode

When the environment ends the episode, at the time limit or for any other reason, and the machine has not ended it itself, the machine charges every item still open: -1 per hard item and -0.5 per soft item, the values `HARD_FAILED_REWARD` and `SOFT_FAILED_REWARD`, counted for each state in `PENDING_PROMISES`. A goal is a hard item, so a task that ends without its goal pays -1 per goal clause, plus -1 for every hard rule still waiting for its answer. The charge is added once, to the reward of the last step, and belongs to no edge of the machine, so no loop can collect it. The charge replaces the flat -0.5 penalty of the time limit; only machines without the list of open items, such as hand-written ones, keep the flat penalty. The well charges the same items, so that breaking a rule never ends the episode more cheaply (Section 4.6.8).

A machine with a final state ends the episode itself when the task is done, and success means reaching that state. A machine without a final state marks as accepting the states where nothing is still owed (`ACCEPTING_STATES`), and an episode that the environment ends in one of them is a success. This differs from LTLf acceptance: a soft rule broken for good counts as accepted, because it already paid -0.5 when it broke, and a hard rule broken for good leads to the well, which rejects on its own. Being in an accepting state never ends the episode.

4.6.8. Breaking a rule never pays better than running out of time

Charging the open items at the end of an episode opens a way to exploit the well. With several items open, running out of time can cost more than breaking a hard rule. In example 1, "Pick up the key, then open the door, then reach the goal", an episode that times out with nothing done pays -3 for its three open goals, while a step into the well pays only -1. An agent close to the time limit could then learn to break a rule on purpose to end the episode more cheaply.

FARM closes this on the edges into the well. Each edge into the well pays -1 plus the charge the episode would get if it ended in the state the edge leaves: -1 per hard item and -0.5 per soft item open there, the broken rule included when it was waiting. Breaking a rule is then always worse than running out of time in that state. In example 1, a break with nothing done costs -4 against -3 for the time limit, and a break from a state with nothing open still costs -1. In example 2, "Reach the goal, and if you drop the key, pick it up right away", the state u1, with the key dropped and the goal still open, costs -2 at the time limit and -3 on the edge into the well; with the flat penalty of the time limit and a plain well, the same two endings paid -0.5 and -1. From u3, the goal reached with the key still owed, the edge into the well costs -2. The same rule settles the case of two hard rules still waiting at the end of a rule-only episode, which would otherwise cost -2 against -1 for a break.

The machine keeps a single well, and the charge sits on each edge into it, so the runtime that executes the machine needs no change. The charge is paid once, on an edge into an absorbing state, so no loop can collect it.

4.6.9. The rewards on one task

Example 2 shows the rewards of this section on one machine. The task is "Reach the goal, and if you drop the key, pick it up right away", a hard Existence(`at_goal`), F `at_goal`, and a hard ChainResponse(`key_lost`, `has_key`), G(`key_lost` $\rightarrow$ X `has_key`). Its machine has five states (Figure 4.10 and Listing G.1), and Table 4.6 gives the to-do list of each.

Table 4.6: The to-do list of each state of example 2. The score is Φ, and the charge is what the machine adds when the environment ends the episode in the state.

State	Still to do	Score	Charge at the end
u0	reach the goal	-0.1	-1
u1	reach the goal, pick up the key at the next step	-0.2	-2
u3	pick up the key at the next step	-0.1	-1
u2, final	nothing	0	—
u4, well	the key was not picked up in time	—	—

A step pays the change in the score. Dropping the key, from u0 to u1, costs -0.1, and picking it up, from u1 back to u0, pays $+0.1$. Reaching the goal pays the $+1$ of the final state plus the progress of the step: 1.1 from u0, and 1.2 from u1 when the key comes back in the same step. Ending the episode in u1 charges -2, one hard item for the goal and one for the key, so the edges from u1 into the well pay $-1-2=-3$, and the edge from u3 pays -2. Table 4.7 adds up six traces with these rewards and with the product-wide potential of the earlier design, which had the flat -0.5 of the time limit and a plain well.

Table 4.7: Rewards of six traces of example 2, step by step. Idle steps pay nothing and are left out. When the environment ends the episode, the last reward includes the charge for the items still open.

	Product-wide		To-do list	
Trace	**Steps**	**Total**	**Steps**	**Total**
Reach the goal	1.2	1.2	1.1	1.1
Drop the key, pick it up, reach the goal	0, 0, 1.2	1.2	−0.1, 0.1, 1.1	1.1
Drop the key, pick it up with the goal	0, 1.2	1.2	−0.1, 1.2	1.1
Nothing happens until the episode ends	−0.5	−0.5	−1	−1
The episode ends right after a drop	−0.5	−0.5	−2.1	−2.1
Drop the key and leave it	0, −1	−1	−0.1, −3	−3.1

With the to-do list, every trace that does the task totals 1.1, with or without a drop, so a drop that is repaired costs nothing in the end. The traces that fail are ordered by what they leave undone. Ending the episode with the goal open costs −1, ending it right after a drop, with the goal and the key open, costs −2.1, and leaving the key on the floor, which breaks the rule, costs −3.1. With the product-wide potential, the drop paid nothing until the rule broke, and an episode that ended right after a drop cost the same −0.5 as one in which nothing happened, so the rewards could not tell the two apart.

The graph reads as in Section 4.6.6: u1 is a state to leave, and picking up the key is the only way out that does not cost. Whether an agent learns this from the rewards is a question for the learning runs (Chapter 7). Appendix H applies the same reading to three more tasks, among them a rule-only task in which one event opens three loops and several events can happen in one step.

4.6.10. Running examples: from the product to the Reward Machine

Figures 4.9, 4.10 and 4.11 draw the Reward Machines that Step 4 builds from the three products. All Reward Machine graphs of the thesis, including panel (c) of Figure F.1, follow one convention. States run from left to right in the order of progress. An arrow without a source marks the initial state, a double circle a final or accepting state and a grey circle the rejecting well. Edges are labelled *guard / reward* as in the products. An edge with more than two guards shows only their number and their reward, for instance *4 guards*, −1, and the listing has the conditions. Implicit stays are not drawn: when no transition of a state matches the event, the machine stays in the state and pays the default reward.

Example 3 shrinks from five states to three. The start p0 and its idle twin p1 merge into u0, and the goal states p3 and p4 become the final state u2, since the only hard clause holds in both. The key is a soft clause, so no state leads to a well. Losing the key costs −0.5 and moves the machine to u1, the former p2. It returns to u0 at the next step without a loss, so each new loss is charged again (Section 4.6.5). The rewards of FL-AT are gone: idle steps pay nothing, and the goal pays 1.1, or 0.6 when the key is lost in the very step of the goal.

Example 2 shrinks from seven states to five. Again p0 and p1 merge into u0. The state p2, where the key is owed, becomes u1, and p4, the goal reached with the key still owed, becomes u3. Only p3 becomes final, as u2: in p4 the hard rule is still pending and blocks the end of the task. The two states where the rule is broken, p5 and p6, become the rejecting well u4, entered with reward −1 when the key is not back at the next step.

The farmed cycle of Step 3 disappears, because the final state ends the episode. It pays 1.2 when the goal is reached with the key in hand and 1.1 when the key comes back after the goal.

Example 1 shrinks from 14 states to five. State u1 is reached after the key, u3 after the key and the door, and u4, the final state, after the goal. Every product state that breaks the order of the task becomes the well u2. Opening the door without the key or reaching the goal too early, which FL-AT still rewarded, now costs -1. Progress pays 0.1 or 0.2, and the goal pays 1.1 to 1.3: the 1 of the final state plus the progress of the step. Listing 4.1 gives the text of this machine, and Appendix G the text of the other two.

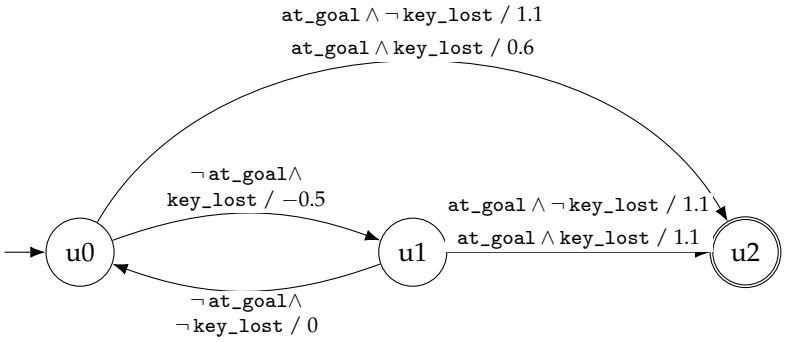


Figure 4.9: The Reward Machine of example 3 after Step 4.

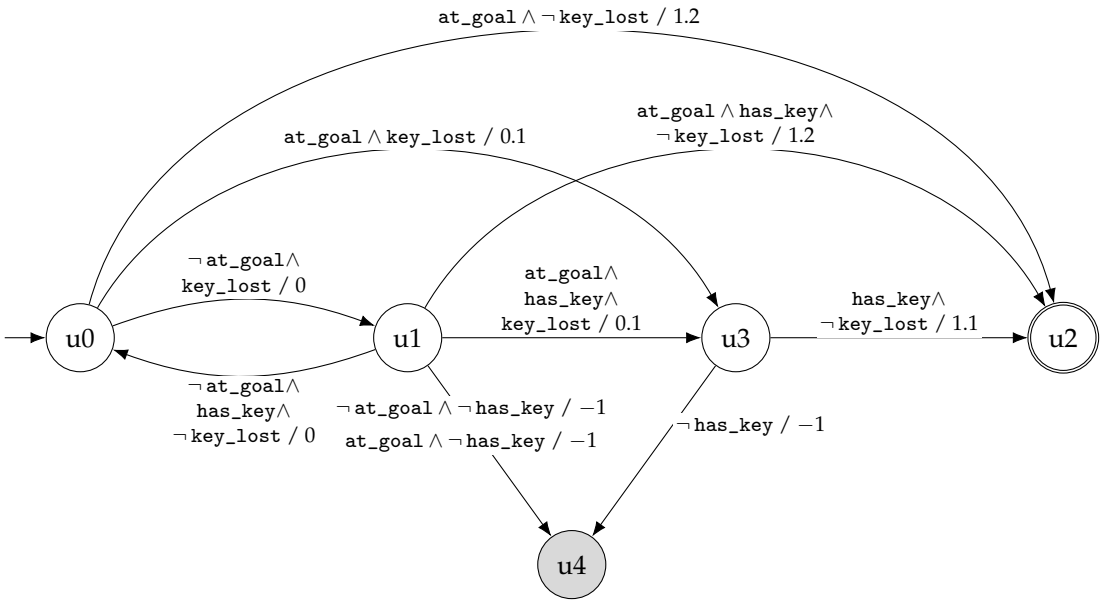


Figure 4.10: The Reward Machine of example 2 after Step 4.

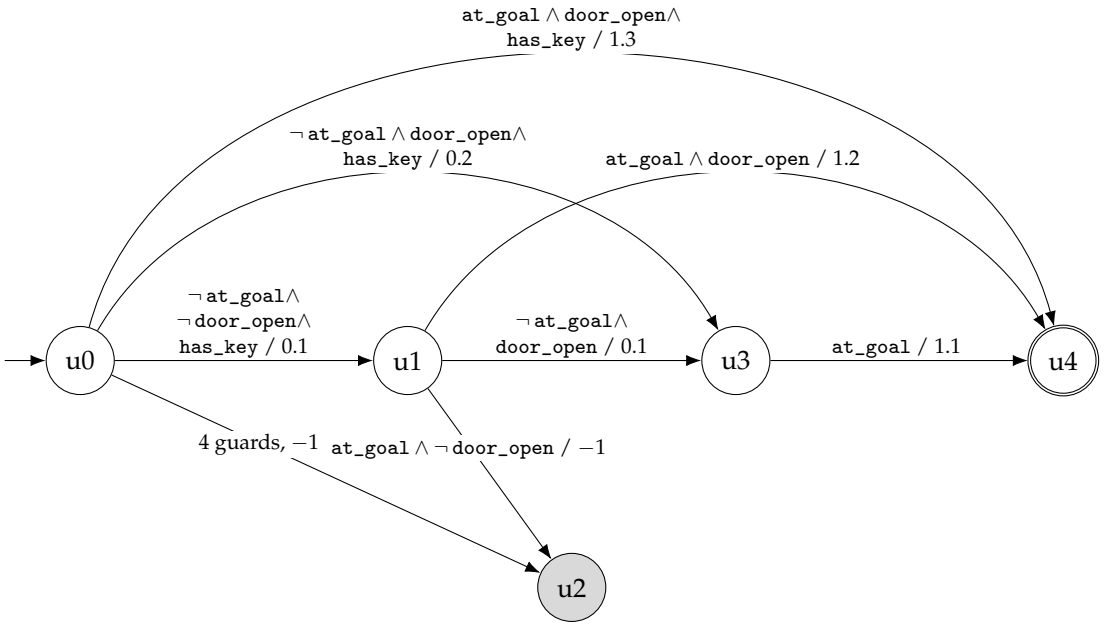


Figure 4.11: The Reward Machine of example 1 after Step 4.

4.7 Step 5 · The critic: review, retries and thinking level

The fifth step shows a second model, the critic, the task, the clauses and the compiled machine together with the warnings of the compiler. The critic accepts the result or flags the clauses it judges wrong, and on a flag the generator tries again with the comments of the critic (Figure 4.12). The critic never edits the machine.

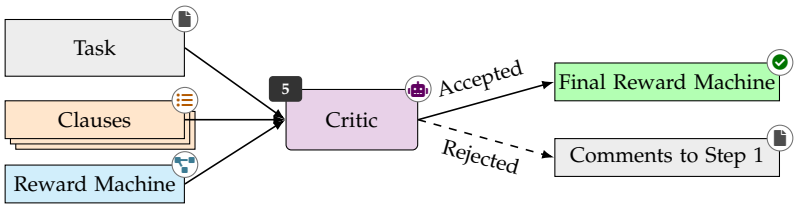


Figure 4.12: Step 5. The critic accepts the machine or sends comments back to the generator.

4.7.1. A review step for the final version

Only the choice of clauses is non-deterministic, so a single review at the end covers the whole pipeline.

4.7.2. LLMs can hallucinate

A critic model can hallucinate as well as the generator [14]. Its verdict therefore only decides whether the generator tries again; the guarantees of the machine come from the compiler (Chapter 3).

4.7.3. Different LLMs for generation and critique

Generator and critic are configured separately. By default the generator is deepseek-v4.1-flash and the critic glm-5.3-flash, so the two roles use different models.

4.7.4. Number of iterations

The generator has three attempts by default, configurable from one to ten. Only rejections by the critic use up an attempt; a failure of the provider is retried without cost.

4.7.5. Fixing mistakes

When the critic flags a clause, the generator receives the comments in its prompt and proposes new clauses, from which the compiler rebuilds the machine.

4.7.6. Flagging clauses whose states cannot occur in the environment

Some clauses lead to machine states that the environment can never reach, for example a door that opens before the key is held. The critic flags these clauses, and the generator retunes them. The critic does not delete states.

4.7.7. Thinking level of the models

A single reasoning level applies to the generator and to the critic. The default is none.

4.7.8. Running examples: the verdict of the critic

The critic accepted the three machines of Step 4 at the first attempt, each time with the verdict `NO CHANGES NEEDED`. The generator therefore did not try again, and the machines of Step 4 are the final output of FARM. A second run of each task gave the same machine (Section D.4).

4.8 The output format

FARM writes each machine in the plain-text format of ARM-FM (Listing 2.1) and extends it. Listing 4.1 shows the machine of the first running example.

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3, u4
INITIAL_STATE: u0
FINAL_STATES: u4
REJECTING_STATES: u2
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !at_goal&!door_open&has_key) -> u1
(u0, !at_goal&door_open&!has_key) -> u2
(u0, !at_goal&door_open&has_key) -> u3
(u0, at_goal&!door_open&!has_key) -> u2
(u0, at_goal&!door_open&has_key) -> u2
(u0, at_goal&door_open&!has_key) -> u2
(u0, at_goal&door_open&has_key) -> u4
(u1, !at_goal&door_open) -> u3
(u1, at_goal&!door_open) -> u2
(u1, at_goal&door_open) -> u4
(u3, at_goal) -> u4
REWARD_FUNCTION:
(u0, !at_goal&!door_open&has_key, u1) -> 0.1
(u0, !at_goal&door_open&!has_key, u2) -> -1
(u0, !at_goal&door_open&has_key, u3) -> 0.2
(u0, at_goal&!door_open&!has_key, u2) -> -1
(u0, at_goal&!door_open&has_key, u2) -> -1
(u0, at_goal&door_open&!has_key, u2) -> -1
(u0, at_goal&door_open&has_key, u4) -> 1.3
(u1, !at_goal&door_open, u3) -> 0.1
(u1, at_goal&!door_open, u2) -> -1
(u1, at_goal&door_open, u4) -> 1.2
(u3, at_goal, u4) -> 1.1
```

Listing 4.1: The Reward Machine FARM generates for the first running example.

The format of FARM differs from that of ARM-FM in five points.

- There are no `else` self-loops. A state with no matching transition stays where it is and pays the default reward.
- The states with a special role are listed explicitly: `FINAL_STATES`, `REJECTING_STATES` and `ACCEPTING_STATES`.
- A single event expression combines propositions with `!` and `&`, such as `has_key & !door_open`, instead of a separate predicate such as `not_has_key`.
- `DEFAULT_REWARD`, `HARD_FAILED_REWARD` and `SOFT_FAILED_REWARD` fix the reward of a transition without an entry and the charge for a failed promise (Section 4.6.7).
- `PENDING_PROMISES` lists, for each state, the hard and soft obligations still open.

Appendix G lists the machine of a task made only of rules, where the last two points apply.

4.9 Details

4.9.1. Contributions

FARM adds the following to the pipeline it builds on.

- A prompt that covers all eight Declare templates and serves any environment, because the propositions and their meaning come from the environment description (Section 4.3).
- A restricted answer: the language model can only choose a template, its propositions and a priority (Section 3.3).
- A fork of FL-AT that returns the state of every clause in each product state (Section 4.5).
- A post-process with hard and soft clauses and a rejecting well, whose rewards treat the task as a to-do list: a step done pays, a new thing to do costs, and the items still open are charged when the environment ends the episode (Section 4.6).
- A critic loop in which the critic flags clauses and the generator retunes them (Section 4.7).

The labelling functions and the state descriptions then follow ARM-FM, which turns the machine into a LARM.

CHAPTER 5
Limitations

5.1 The task is assumed to be achievable

5.2 Contradictions

5.3 Unknown facts

5.4 Shortest path

5.5 Human language can be misleading

CHAPTER 6
Evaluation methodology

The baseline is ARM-FM as described in its paper. No official code is available, so the baseline is rebuilt from the appendices of the paper and, for the MiniGrid tasks, from the prompts and machines in the first author's public repository [15]. Parts that neither source documents, such as the framing of later generator and critic rounds, are reconstructed. Requests to the authors for further material went unanswered, so the rebuilt baseline may differ from the original and the results compare FARM with this reconstruction.

6.1 Empirical evaluation in a real environment

The learning runs train the same learner on the same tasks with different machines and compare the agents. The comparison with ARM-FM trains with the machines of FARM and with those of the rebuilt baseline, and reports the success rate and the number of steps needed to reach it. The reward design of Step 4 is measured on tasks whose rules wait for an event, such as example 2: the agent trains with the to-do list and with the product-wide potential of the earlier design (Section 4.6), and the runs report how often the agent makes the repair that the rule asks for and how many steps the repair takes.

6.2 Qualitative evaluation of the Reward Machines

6.2.1. From traces to rewards

6.2.2. Shape, number of states and useless states

CHAPTER 7
Results

7.1 Comparison between ARM-FM and FARM

7.1.1. Learning with the machines

Evidence pending. The learning runs of Section 6.1: success rate and steps to reach it, FARM against the rebuilt ARM-FM.

If the evidence supports it: Agents trained with the machines of FARM reach a success rate of [rate] after [steps] steps, against [rate] with the machines of the rebuilt ARM-FM, and they reach the success rate of ARM-FM in [share] fewer steps. On these tasks, machines that follow the task by construction also teach it faster.

If it does not: Agents trained with the machines of FARM reach a success rate of [rate], against [rate] with the machines of the rebuilt ARM-FM, and they learn no faster. On these tasks the gain of FARM lies in the machine itself: it follows the task by construction, gives every event a price and pays no loop (Section 4.6), but the learner does not turn this into faster learning.

7.2 Reacting to events

Evidence pending. The learning runs of Section 6.1 on tasks whose rules wait for an event, such as example 2: how often the agent makes the repair, to-do list against the product-wide potential.

If the evidence supports it: With the to-do list, trained agents pick the key up again after [share] of the drops, [steps] steps after the drop on average, against [share] with the product-wide potential. The penalty at the event and the payback at the repair are enough for the agent to learn the reaction that the rule asks for.

If it does not: With the to-do list, trained agents pick the key up again after [share] of the drops, no more often than with the product-wide potential ([share]). The machine prices every event as Section 4.6.9 shows, but in these runs a penalty of -0.1 at the event and the charge at the end of the episode do not teach the agent to react. Why, and what would, is left for future work (Chapter 9).

CHAPTER 8
Conclusions

The rewards of FARM treat the task as a to-do list, so the transitions of the machine price what happens in the environment and what the instructions ask in return. On the machines of this thesis, a step towards the task or towards a rule pays, and a broken rule or a new thing to do costs. A step out of the order of the task leads to the well, the same event costs the same wherever it happens, a break that is repaired pays back what it cost, breaking a rule costs more than running out of time, and no loop pays (Section 4.6.9 and Appendix H).

Evidence pending. The learning runs of Section 7.1.1.

If the evidence supports it: Agents trained with the machines of FARM learn the tasks faster than with those of the rebuilt ARM-FM. Machines built from the logic of the task also teach it faster.

If it does not: Agents trained with the machines of FARM learn the tasks no faster than with those of the rebuilt ARM-FM. The gain that this thesis shows lies in the machines: they follow the task by construction, and their rewards can be read and checked.

Evidence pending. The reaction runs of Section 7.2.

If the evidence supports it: Agents also learn from these rewards how to react. The reward tells the agent what is good and what is wrong, what it must still do, how it must react to an event and in which order it must finish the task, and the agent follows it.

If it does not: Whether a learner turns these rewards into reactions remains open: in the runs of this thesis, agents do not repair events more often than with the earlier design. The rewards are consistent and cannot be farmed, which the checks on the machines show, but that does not yet make the agent react.

CHAPTER 9
Future work

Bibliography

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. Adaptive Computation and Machine Learning series. MIT Press, 2018. ISBN: 9780262039246.

[2] Rodrigo Toro Icarte et al. "Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning". In: *Journal of Artificial Intelligence Research* 73 (Jan. 2022), pp. 173–208. ISSN: 1076-9757. DOI: 10.1613/jair.1.12440.

[3] Andrew Y. Ng, Daishi Harada, and Stuart Russell. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping". In: *International Conference on Machine Learning*. 1999, pp. 278–287. URL: https://mlanthology.org/icml/1999/ng1999icml-policy/.

[4] Rodrigo Toro Icarte et al. "Using Reward Machines for High-Level Task Specification and Decomposition in Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, Oct. 2018, pp. 2107–2116. URL: https://proceedings.mlr.press/v80/icarte18a.html.

[5] Roger Creus Castanyer et al. "ARM-FM: Automated Reward Machines via Foundation Models for Compositional Reinforcement Learning". In: *International Conference on Learning Representations*. Published at ICLR 2026. 2026. URL: https://openreview.net/forum?id=OBpQdCWLfd.

[6] Amir Pnueli. "The temporal logic of programs". In: *18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*. IEEE, Sept. 1977, pp. 46–57. DOI: 10.1109/SFCS.1977.32.

[7] Giuseppe De Giacomo and Moshe Y. Vardi. "Linear Temporal Logic and Linear Dynamic Logic on Finite Traces". In: *International Joint Conference on Artificial Intelligence*. 2013, pp. 854–860. URL: https://mlanthology.org/ijcai/2013/giacomo2013ijcai-linear/.

[8] Jesper G. Henriksen et al. "Mona: Monadic second-order logic in practice". In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer Berlin Heidelberg, 1995, pp. 89–110. DOI: 10.1007/3-540-60630-0_5.

[9] Alberto Camacho et al. "LTL and Beyond: Formal Languages for Reward Function Specification in Reinforcement Learning". In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, Aug. 2019, pp. 6065–6073. DOI: 10.24963/ijcai.2019/840.

[10] M. Pesic and W. M. P. van der Aalst. "A Declarative Approach for Flexible Business Processes Management". In: *Business Process Management Workshops*. Springer Berlin Heidelberg, 2006, pp. 169–180. DOI: 10.1007/11837862_18.

[11] Francesco Fuggitti and Tathagata Chakraborti. "NL2LTL – a Python Package for Converting Natural Language (NL) Instructions to Linear Temporal Logic (LTL) Formulas". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.13 (June 2023), pp. 16428–16430. ISSN: 2159-5399. DOI: 10.1609/aaai.v37i13.27068.

[12] A. Gerevini and D. Long. "Preferences and soft constraints in PDDL3". In: *ICAPS Workshop on Planning with Preferences and Soft Constraints*. Ed. by A. Gerevini and D. Long. June 2006, pp. 46–53. URL: https://strathprints.strath.ac.uk/3149/.

[13] Shuyin Ouyang et al. "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation". In: *ACM Transactions on Software Engineering and Methodology* 34.2 (Jan. 2025), pp. 1–28. ISSN: 1557-7392. DOI: 10.1145/3697010.

[14] Lei Huang et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* 43.2 (Jan. 2025), pp. 1–55. ISSN: 1558-2868. DOI: 10.1145/3703155.

[15] Roger Creus. *llms-rm-rl*. 2025. URL: https://github.com/roger-creus/llms-rm-rl (visited on 10/27/2026).

[16] Jiaming Ji et al. *AI Alignment: A Comprehensive Survey*. 2025. arXiv: 2310.19852 [cs.AI]. URL: https://arxiv.org/abs/2310.19852.

[17] Jaime Middleton et al. "FL-AT: A Formal Language–Automaton Transmogrifier". In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. System Demo. 2020.

[18] IBM. *nl2ltl: Natural Language (NL) to Linear Temporal Logic (LTL)*. 2022. URL: https://github.com/IBM/nl2ltl (visited on 10/27/2026).

[19] Jaime Middleton. *Flat*. 2019. URL: https://github.com/Jamidd/Flat (visited on 10/27/2026).

APPENDIX A

Sustainable Development Goals

APPENDIX B

AI usage

APPENDIX C

Development tools

APPENDIX D
Implementation details

D.1 IBM nl2ltl

FARM uses a fork of IBM's nl2ltl (https://github.com/MiquelGomezCorral/nl2ltl), which is forked from https://github.com/IBM/nl2ltl [18]. The GPT engine of nl2ltl is bypassed. FARM supplies its own engine, which asks a language model for a list of clauses under a JSON schema and passes each clause to the Declare template classes of nl2ltl, which build the LTLf formula. Only the template classes and the LTLf builders are reused. The change in the fork concerns the build: it supports modern Python packaging and drops the pin on the OpenAI client, which the new engine does not need.

D.2 FL-AT

FARM uses a fork of FL-AT (https://github.com/MiquelGomezCorral/Flat), which is forked from https://github.com/Jamidd/Flat [19]. The fork makes five changes.

- Library packaging: the code installs as a package and uses package-relative imports.
- MONA runs in a subprocess with temporary files, where the original wrote a file in the working folder through `os.system`, and its output is parsed strictly.
- Proposition names are renamed with regular expressions, where the original split strings and evaluated labels.
- MONA's state 0, the do-nothing start state of Section 4.4.2, is removed.
- The state tuples of the product are exported, so that each product state tells which state every clause is in.

D.3 Models and settings

Unless stated otherwise, generation uses the defaults of the configuration: the OpenCode provider, deepseek-v4.1-flash as generator, glm-5.3-flash as critic, reasoning level none and at most three attempts. The running examples did not use these defaults. The monthly quota of the OpenCode plan was exhausted on 29 October 2026, so they were produced with the Antigravity provider (Section D.4).

D.4 How the examples were produced

The four running examples (Section 4.2.1 and Appendix G) were generated on 29 October 2026 with the Antigravity provider (`agy` 1.4.10). The generator was gemini-3.8-flash-medium and the critic gpt-oss-120b-medium. The effort level is part of the model name, so the reasoning setting stays at none. Each task ran with at most three attempts, and the critic accepted every proposal at the first one. A second run of each task gave an identical Reward Machine.

The code is commit a56bbed of the branch `feat/rule-only-rewards`, plus a fix to the Antigravity engine that was not yet committed (branch `fix/antigravity-structured-output`). The fix is needed because `agy` enforces a JSON schema only through its built-in `finish` tool, which the agent without tools cannot call. The engine now places the schema in the instructions and validates the reply itself.

Each running example is one run of the command-line interface on the DoorKey environment description with a single task:

```
LLM_PROVIDER=antigravity \
ANTIGRAVITY_MODEL_GENERATOR=gemini-3.8-flash-medium \
ANTIGRAVITY_MODEL_CRITIC=gpt-oss-120b-medium \
python app/main.py generate-rm \
  --environment examples/arm-fm/minigrid-doorkey/environment.md \
  --task "<task>" --output rm.txt --overwrite
```

A small driver script that builds the same configuration and only redirects the output folders ran the examples. A script draws the Reward Machine graphs of Section 4.6.10 and Appendix G from the text of each machine, parsed with the repository's own parser, and panel (c) of Figure F.1 uses the same drawing code. A second script draws the DFAs and products of Sections 4.4.3 and 4.5.4 from the clauses of the same runs, with MONA and the FL-AT fork and no further call to a language model. The MONA figures of Appendix F need no language model: they come from the compiler run on one fixed clause.

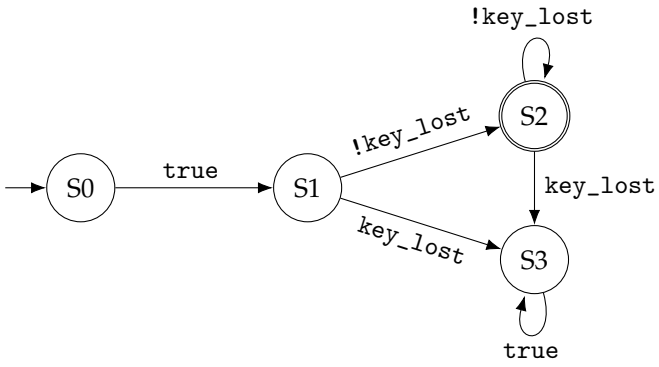
APPENDIX E

IBM template examples

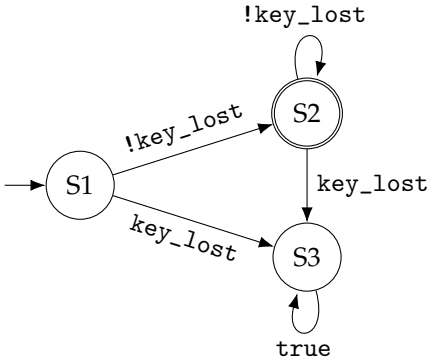
APPENDIX F

MONA automata before and after cleaning

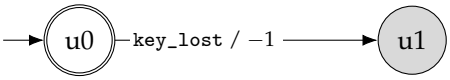
Figure F.1 follows one clause, Absence(`key_lost`) with the LTLf formula $\neg$ F `key_lost`, through the two removals of redundant states described in Section 4.4.2. Panel (a) is the automaton that MONA prints. Its start state S0 moves to S1 on every input. Panel (b) is the same automaton after the FL-AT fork removes S0. In panel (c) the post-process has merged the "no step yet" start S1 into its idle twin S2, which behaves identically on every input, and has priced the edges. The clause is kept hard in panel (c), and the machine is the one that the reward layer builds from this clause alone. Panels (a) and (b) draw every transition of the automaton, and double circles mark accepting states. Panel (c) follows the convention of Section 4.6.10: the rejecting well u1 is grey and is entered with reward -1, and the implicit stay of u0 is not drawn. Running example 3 (Section 4.6.10) contains this clause as a soft one, while the figure keeps it hard to show the well, so the machine of that example differs from panel (c).



(a) Raw MONA automaton, with state S0.



(b) After the FL-AT fork removes state S0.



(c) After the post-process merges the "no step yet" start into its idle twin.

Figure F.1: The automaton of Absence(`key_lost`) before and after the two removals of redundant states.

Listing F.1 gives the text of the machine in panel (c). The implicit stay of u0 is not listed, and there are no pending promises, since the only promise is hard and its violation leads to the well.

```
REWARD_MACHINE:
STATES: u0, u1
INITIAL_STATE: u0
FINAL_STATES:
REJECTING_STATES: u1
ACCEPTING_STATES: u0
DEFAULT_REWARD: 0
HARD_FAILED_REWARD: -1
SOFT_FAILED_REWARD: -0.5
TRANSITION_FUNCTION:
(u0, key_lost) -> u1
REWARD_FUNCTION:
(u0, key_lost, u1) -> -1
PENDING_PROMISES:
```

Listing F.1: The Reward Machine of the hard clause Absence(`key_lost`).

APPENDIX G
Running examples

This appendix gives the text of the Reward Machines of running examples 2 and 3, whose graphs are in Section 4.6.10, and example 4 in full, for the DoorKey environment. The machine of example 1 is Listing 4.1. The texts and graphs are the unedited outputs of the runs described in Section D.4, and the graphs follow the convention of Section 4.6.10.

G.1 Example 2: a hard rule

Task. Reach the goal, and if you drop the key, pick it up right away.

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3, u4
INITIAL_STATE: u0
FINAL_STATES: u2
REJECTING_STATES: u4
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !at_goal&key_lost) -> u1
(u0, at_goal&!key_lost) -> u2
(u0, at_goal&key_lost) -> u3
(u1, !at_goal&!has_key) -> u4
(u1, !at_goal&has_key&!key_lost) -> u0
(u1, at_goal&!has_key) -> u4
(u1, at_goal&has_key&!key_lost) -> u2
(u1, at_goal&has_key&key_lost) -> u3
(u3, !has_key) -> u4
(u3, has_key&!key_lost) -> u2
REWARD_FUNCTION:
(u0, at_goal&!key_lost, u2) -> 1.2
(u0, at_goal&key_lost, u3) -> 0.1
(u1, !at_goal&!has_key, u4) -> -1
(u1, at_goal&!has_key, u4) -> -1
(u1, at_goal&has_key&!key_lost, u2) -> 1.2
(u1, at_goal&has_key&key_lost, u3) -> 0.1
(u3, !has_key, u4) -> -1
(u3, has_key&!key_lost, u2) -> 1.1
```

Listing G.1: The Reward Machine of example 2.

G.2 Example 3: a soft preference

Task. Reach the goal, ideally without ever losing the key.

```
REWARD_MACHINE:
STATES: u0, u1, u2
INITIAL_STATE: u0
FINAL_STATES: u2
DEFAULT_REWARD: 0
TRANSITION_FUNCTION:
(u0, !at_goal&key_lost) -> u1
(u0, at_goal&!key_lost) -> u2
(u0, at_goal&key_lost) -> u2
(u1, !at_goal&!key_lost) -> u0
(u1, at_goal&!key_lost) -> u2
(u1, at_goal&key_lost) -> u2
REWARD_FUNCTION:
(u0, !at_goal&key_lost, u1) -> -0.5
(u0, at_goal&!key_lost, u2) -> 1.1
(u0, at_goal&key_lost, u2) -> 0.6
(u1, at_goal&!key_lost, u2) -> 1.1
(u1, at_goal&key_lost, u2) -> 1.1
```

Listing G.2: The Reward Machine of example 3.

G.3 Example 4: a task made only of rules

Task. If you drop the key, pick it up again.

Clause	Template	Priority	LTLf
Response(key_lost, has_key)	Response(key_lost, has_key)	hard	G (key_lost → F has_key)

The only clause is a rule, so the machine has no goal and no final state (Listing G.3 and Figure G.1). State u0 is accepting: nothing is owed there. State u1 owes the key, and PENDING_PROMISES lists it with one hard promise. If the environment ends the episode in u1, the machine charges -1 once, the value HARD_FAILED_REWARD. In u0 it charges nothing. Dropping the key costs -0.1 and picking it up returns $+0.1$, so the two shaping rewards cancel over the cycle.

```
REWARD_MACHINE:
STATES: u0, u1
INITIAL_STATE: u0
FINAL_STATES:
ACCEPTING_STATES: u0
DEFAULT_REWARD: 0
HARD_FAILED_REWARD: -1
SOFT_FAILED_REWARD: -0.5
TRANSITION_FUNCTION:
(u0, !has_key&key_lost) -> u1
(u1, has_key) -> u0
REWARD_FUNCTION:
(u0, !has_key&key_lost, u1) -> -0.1
(u1, has_key, u0) -> 0.1
PENDING_PROMISES:
u1: hard 1, soft 0
```

Listing G.3: The Reward Machine of example 4, a task made only of rules.

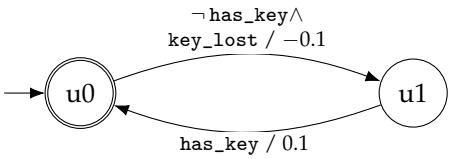


Figure G.1: The graph of the Reward Machine of example 4.

APPENDIX H
The to-do list on more tasks

This appendix reads the rewards of three more machines in the way Section 4.6.9 reads example 2. In each task the product-wide potential of the earlier design left an event loop unpaid or priced the same event in different ways. The tables set that potential, with the flat -0.5 of the time limit and a plain well, against the to-do list of Section 4.6. A step pays the change in the score: $+0.1$ for each item crossed off and -0.1 for each item added.

H.1 An alarm and a power cut

Task. When the alarm rings, switch it off at the very next step. Preferably, if the power ever goes out, run the generator.

Clause	Priority	LTLf
ChainResponse(`alarm_rings`, `alarm_off`)	hard	$\mathrm{G}\,(\texttt{alarm_rings} \rightarrow \mathrm{X}\,\texttt{alarm_off})$
RespondedExistence(`power_cut`, `generator_on`)	soft	$\mathrm{F}\,\texttt{power_cut} \rightarrow \mathrm{F}\,\texttt{generator_on}$

The task has no goal, so the machine has no final state (Listing H.1). This machine holds several cases at once: three loops on the same event, a well, and steps in which two or three events happen at once. Table H.1 gives the to-do list of its seven states.

Table H.1: The to-do list of each state of the alarm machine.

State	Still to do	Score	Charge at the end
u0, accepting	nothing	0	0
u1	run the generator (soft)	-0.1	-0.5
u2, accepting	nothing, the generator ran	0	0
u3	switch the alarm off (hard)	-0.1	-1
u4	switch the alarm off, run the generator	-0.2	-1.5
u5	switch the alarm off, the generator ran	-0.1	-1
u6, well	the alarm was not switched off in time	—	—

The alarm can ring in u0, u1 and u2, and each time it adds one item. The ring costs -0.1 and switching the alarm off at the next step pays $+0.1$, in each of the three loops u0–u3, u1–u4 and u2–u5. The product-wide potential priced the same ring in three ways: $-0.3/+0.3$ in u0–u3, $-0.1/+0.1$ in u2–u5 and 0 in u1–u4. Every loop nets 0 under both designs, so no loop pays, but only the to-do list gives the ring one price.

Several events can happen in one step, and their prices add up (Table H.2). Switching the alarm off as the power goes out crosses one item off and adds another, and pays 0; the product-wide potential paid $+0.1$ for it.

Table H.2: Steps of the alarm machine in which several events happen at once.

Step	What happens	Items	Product-wide	To-do list
u3 → u1	alarm off, power goes out	$-1, +1$	$+0.1$	0
u3 → u4	alarm off, rings again, power goes out	$-1, +2$	$+0.1$	-0.1
u3 → u5	alarm off, rings again, generator on	$-1, +1$	$+0.2$	0
u3 → u2	alarm off, generator on	-1	$+0.3$	$+0.1$
u1 → u5	alarm rings, generator on	$-1, +1$	$+0.1$	0
u0 → u4	alarm rings, power goes out	$+2$	-0.2	-0.2
u4 → u2	alarm off, generator on	-2	$+0.2$	$+0.2$
u4 → u5	alarm off, rings again, generator on	$-2, +1$	$+0.1$	$+0.1$

The edges into the well pay -1 plus the charge of the state they leave (Section 4.6.8): -2 from u3 and from u5, and -2.5 from u4, where the generator is also owed. With a plain well they all paid -1. Not switching the alarm off now costs more than letting the time run out in the same state.

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3, u4, u5, u6
INITIAL_STATE: u0
FINAL_STATES:
REJECTING_STATES: u6
ACCEPTING_STATES: u0, u2
DEFAULT_REWARD: 0
HARD_FAILED_REWARD: -1
SOFT_FAILED_REWARD: -0.5
TRANSITION_FUNCTION:
(u0, !alarm_rings&!generator_on&power_cut) -> u1
(u0, !alarm_rings&generator_on) -> u2
(u0, alarm_rings&!generator_on&!power_cut) -> u3
(u0, alarm_rings&!generator_on&power_cut) -> u4
(u0, alarm_rings&generator_on) -> u5
(u1, !alarm_rings&generator_on) -> u2
(u1, alarm_rings&!generator_on) -> u4
(u1, alarm_rings&generator_on) -> u5
(u2, alarm_rings) -> u5
(u3, !alarm_off&!generator_on&!power_cut) -> u6
(u3, !alarm_off&!generator_on&power_cut) -> u6
(u3, !alarm_off&generator_on) -> u6
(u3, alarm_off&!alarm_rings&!generator_on&!power_cut)
    ↪ -> u0
(u3, alarm_off&!alarm_rings&!generator_on&power_cut)
    ↪ -> u1
(u3, alarm_off&!alarm_rings&generator_on) -> u2
(u3, alarm_off&alarm_rings&!generator_on&power_cut) ->
    ↪  u4
(u3, alarm_off&alarm_rings&generator_on) -> u5
(u4, !alarm_off&!generator_on) -> u6
(u4, !alarm_off&generator_on) -> u6
(u4, alarm_off&!alarm_rings&!generator_on) -> u1
(u4, alarm_off&!alarm_rings&generator_on) -> u2
(u4, alarm_off&alarm_rings&generator_on) -> u5
```

```
(u5, !alarm_off) -> u6
(u5, alarm_off&!alarm_rings) -> u2
REWARD_FUNCTION:
(u0, !alarm_rings&!generator_on&power_cut, u1) -> -0.1
(u0, alarm_rings&!generator_on&!power_cut, u3) -> -0.1
(u0, alarm_rings&!generator_on&power_cut, u4) -> -0.2
(u0, alarm_rings&generator_on, u5) -> -0.1
(u1, !alarm_rings&generator_on, u2) -> 0.1
(u1, alarm_rings&!generator_on, u4) -> -0.1
(u2, alarm_rings, u5) -> -0.1
(u3, !alarm_off&!generator_on&!power_cut, u6) -> -2
(u3, !alarm_off&!generator_on&power_cut, u6) -> -2
(u3, !alarm_off&generator_on, u6) -> -2
(u3, alarm_off&!alarm_rings&!generator_on&!power_cut,
    ↪ u0) -> 0.1
(u3, alarm_off&!alarm_rings&generator_on, u2) -> 0.1
(u3, alarm_off&alarm_rings&!generator_on&power_cut, u4
    ↪ ) -> -0.1
(u4, !alarm_off&!generator_on, u6) -> -2.5
(u4, !alarm_off&generator_on, u6) -> -2.5
(u4, alarm_off&!alarm_rings&!generator_on, u1) -> 0.1
(u4, alarm_off&!alarm_rings&generator_on, u2) -> 0.2
(u4, alarm_off&alarm_rings&generator_on, u5) -> 0.1
(u5, !alarm_off, u6) -> -2
(u5, alarm_off&!alarm_rings, u2) -> 0.1
PENDING_PROMISES:
u1: hard 0, soft 1
u3: hard 1, soft 0
u4: hard 1, soft 1
u5: hard 1, soft 0
```

Listing H.1: The Reward Machine of the alarm task.

H.2 A box that falls off the shelf, and a key

Task. Whenever the box falls off the shelf, put it back. Preferably pick up the key at some point.

Clause	Priority	LTLf
Response(`box_fell`, `box_on_shelf`)	hard	G (`box_fell` → F `box_on_shelf`)
Existence(`has_key`)	soft	F `has_key`

The only hard clause is a Response, which never breaks for good but only waits, and the only goal is soft, so the machine has neither a well nor a final state (Listing H.2). Leaving the box on the floor costs the -0.1 of the fall, never paid back, and -1 when the episode ends.

Table H.3: The to-do list of each state of the box and key machine.

State	Still to do	Score	Charge at the end
u0	pick up the key (soft)	-0.1	-0.5
u1, accepting	nothing	0	0
u2	put the box back, pick up the key	-0.2	-1.5
u3	put the box back (hard)	-0.1	-1

Table H.4: The edges of the box and key machine that pay a reward.

Edge	What happens	Product-wide	To-do list
u0 → u1	key picked up	+0.2	+0.1
u0 → u2	box falls	0	−0.1
u0 → u3	box falls, key picked up	+0.1	0
u1 → u3	box falls	−0.1	−0.1
u2 → u0	box put back	0	+0.1
u2 → u1	box put back, key picked up	+0.2	+0.2
u2 → u3	key picked up	+0.1	+0.1
u3 → u1	box put back	+0.1	+0.1

The product-wide potential priced the fall of the box by whether the key was already held: the loop u0–u2, before the key, paid nothing, and the loop u1–u3, after it, paid −0.1/+0.1. The to-do list pays −0.1/+0.1 in both (Table H.4), because the fall adds the same item in both.

```
REWARD_MACHINE:
STATES: u0, u1, u2, u3
INITIAL_STATE: u0
FINAL_STATES:
ACCEPTING_STATES: u1
DEFAULT_REWARD: 0
HARD_FAILED_REWARD: -1
SOFT_FAILED_REWARD: -0.5
TRANSITION_FUNCTION:
(u0, !box_fell&has_key) -> u1
(u0, box_fell&!box_on_shelf&!has_key) -> u2
(u0, box_fell&!box_on_shelf&has_key) -> u3
(u0, box_fell&box_on_shelf&has_key) -> u1
(u1, box_fell&!box_on_shelf) -> u3
(u2, !box_on_shelf&has_key) -> u3
(u2, box_on_shelf&!has_key) -> u0
(u2, box_on_shelf&has_key) -> u1
(u3, box_on_shelf) -> u1
REWARD_FUNCTION:
(u0, !box_fell&has_key, u1) -> 0.1
(u0, box_fell&!box_on_shelf&!has_key, u2) -> -0.1
(u0, box_fell&box_on_shelf&has_key, u1) -> 0.1
(u1, box_fell&!box_on_shelf, u3) -> -0.1
(u2, !box_on_shelf&has_key, u3) -> 0.1
(u2, box_on_shelf&!has_key, u0) -> 0.1
(u2, box_on_shelf&has_key, u1) -> 0.2
(u3, box_on_shelf, u1) -> 0.1
PENDING_PROMISES:
u0: hard 0, soft 1
u2: hard 1, soft 1
u3: hard 1, soft 0
```

Listing H.2: The Reward Machine of the box and key task.

H.3 A door and a box

Task. Open the door. Whenever the box falls off the shelf, preferably put it back.

Clause	Priority	LTLf
Existence(door_open)	hard	F door_open
Response(box_fell, box_on_shelf)	soft	G (box_fell → F box_on_shelf)

The machine has three states (Listing H.3): u0 owes the door, with score −0.1 and charge −1; u2 owes the door and the box, with score −0.2 and charge −1.5; and u1 is final. The loop u0–u2, where the box falls and is put back, paid nothing with the product-wide potential and pays −0.1/+0.1 with the to-do list. Table H.5 adds up six traces.

Table H.5: Rewards of six traces of the door and box task, step by step. Idle steps are left out; when the environment ends the episode, the last reward includes the charge for the items still open.

	Product-wide		To-do list	
Trace	**Steps**	**Total**	**Steps**	**Total**
Nothing happens until the episode ends	−0.5	−0.5	−1	−1
The box falls and stays down	0, −0.5	−0.5	−0.1, −1.5	−1.6
The box falls and is put back	0, 0, −0.5	−0.5	−0.1, 0.1, −1	−1
Open the door	1.1	1.1	1.1	1.1
The box falls, the door opens	0, 0.6	0.6	−0.1, 0.7	0.6
The box falls, is put back as the door opens	0, 1.1	1.1	−0.1, 1.2	1.1

With the product-wide potential and the flat −0.5, an episode that ended with the box on the floor cost the same as one in which the box was put back, so at the time limit the preference changed nothing. With the to-do list the first costs −1.6 and the second −1. When the door opens with the box still on the floor, the finishing edge pays $1 + 0.2 - 0.5 = 0.7$, where −0.5 is the charge of a soft clause still pending, and the trace totals 0.6 under both designs.

```
REWARD_MACHINE:
STATES: u0, u1, u2
INITIAL_STATE: u0
FINAL_STATES: u1
ACCEPTING_STATES:
DEFAULT_REWARD: 0
HARD_FAILED_REWARD: -1
SOFT_FAILED_REWARD: -0.5
TRANSITION_FUNCTION:
(u0, !box_fell&door_open) -> u1
(u0, box_fell&!box_on_shelf&!door_open) -> u2
(u0, box_fell&!box_on_shelf&door_open) -> u1
(u0, box_fell&box_on_shelf&door_open) -> u1
(u2, !box_on_shelf&door_open) -> u1
(u2, box_on_shelf&!door_open) -> u0
(u2, box_on_shelf&door_open) -> u1
REWARD_FUNCTION:
(u0, !box_fell&door_open, u1) -> 1.1
(u0, box_fell&!box_on_shelf&!door_open, u2) -> -0.1
(u0, box_fell&!box_on_shelf&door_open, u1) -> 0.6
(u0, box_fell&box_on_shelf&door_open, u1) -> 1.1
(u2, !box_on_shelf&door_open, u1) -> 0.7
```

```
(u2, box_on_shelf&!door_open, u0) -> 0.1
(u2, box_on_shelf&door_open, u1) -> 1.2
PENDING_PROMISES:
u0: hard 1, soft 0
u2: hard 1, soft 1
```

Listing H.3: The Reward Machine of the door and box task.

APPENDIX I
Formatting examples

This appendix collects formatting examples for figures, tables, plots and algorithms. It will be removed before submission.

Figure I.1 shows a single image.

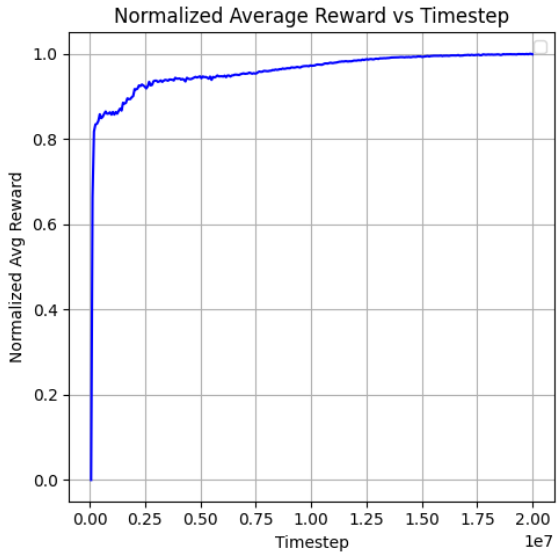
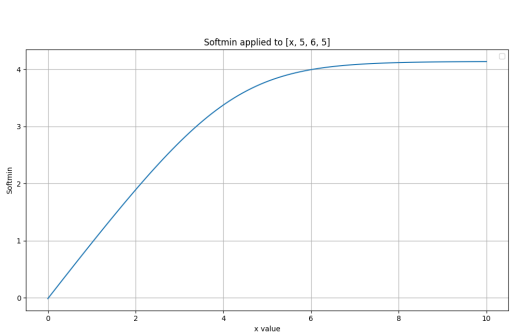
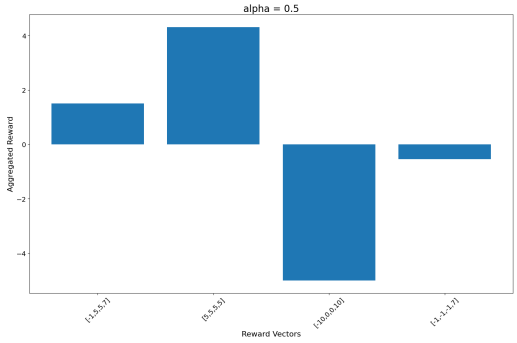


Figure I.1: Example of a single image.

Figure I.2 shows two subfigures side by side, Figures I.2a and I.2b.



(a) First subfigure.



(b) Second subfigure.

Figure I.2: Example of two subfigures side by side.

Figure I.3 shows a diagram drawn with TikZ.

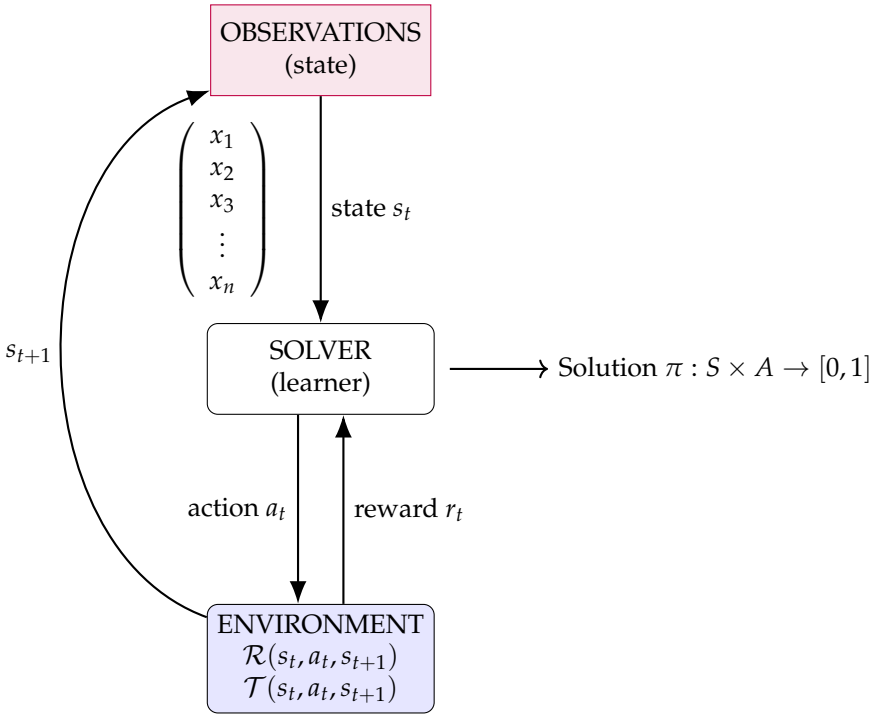


Figure I.3: Example of a TikZ diagram.

Figure I.4 shows a multi-panel plot drawn with `pgfplots`.

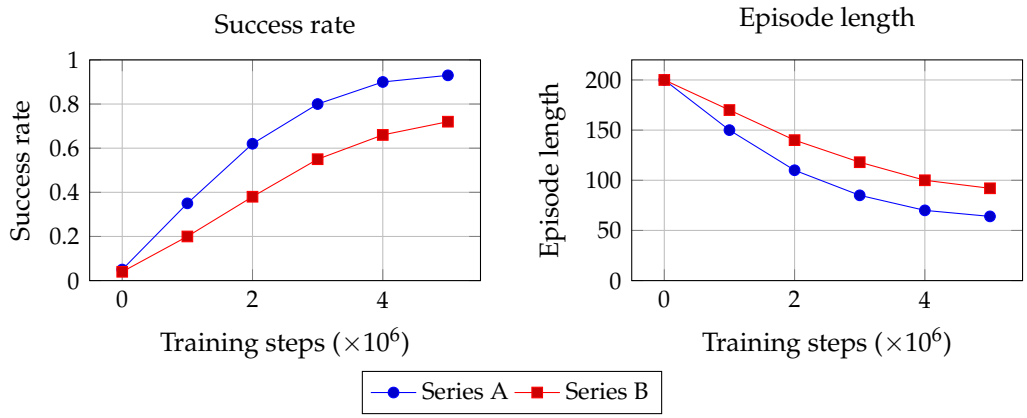


Figure I.4: Example of a two-panel plot with illustrative data: success rate and episode length against training steps for two series.

Table I.1 shows a table in `booktabs` format.

Table I.1: Example of a `booktabs` table with illustrative data.

Series	Success rate	Episode length
Series A	0.93	64
Series B	0.72	92

Algorithm I.1 shows a block of pseudocode.

Algorithm I.1: Example of a pseudocode block

Input: list of items L, threshold θ
Output: list of kept items L'

```
1:  procedure FILTER(L, θ)
2:      L' ← ∅
3:      for each x ∈ L do
4:          if f(x) > θ then
5:              L' ← L' ∪ {x}                ▷ keep the item
6:          else
7:              continue                     ▷ skip the item
8:          end if
9:      end for
10:     return L'
11: end procedure
```